

PRIORITIZED EXPERIENCE REPLAY

Tom Schaul, John Quan, Ioannis Antonoglou and David Silver

Google DeepMind

{schaul, johnquan, ioannisa, davidsilver}@google.com

ABSTRACT

Experience replay lets online reinforcement learning agents remember and reuse experiences from the past. In prior work, experience transitions were uniformly sampled from a replay memory. However, this approach simply replays transitions at the same frequency that they were originally experienced, regardless of their significance. In this paper we develop a framework for prioritizing experience, so as to replay important transitions more frequently, and therefore learn more efficiently. We use prioritized experience replay in Deep Q-Networks (DQN), a reinforcement learning algorithm that achieved human-level performance across many Atari games. DQN with prioritized experience replay achieves a new state-of-the-art, outperforming DQN with uniform replay on 41 out of 49 games.

1 INTRODUCTION

Online reinforcement learning (RL) agents incrementally update their parameters (of the policy, value function or model) while they observe a stream of experience. In their simplest form, they discard incoming data immediately, after a single update. Two issues with this are (a) strongly correlated updates that break the i.i.d. assumption of many popular stochastic gradient-based algorithms, and (b) the rapid forgetting of possibly rare experiences that would be useful later on.

Experience replay (Lin, 1992) addresses both of these issues: with experience stored in a replay memory, it becomes possible to break the temporal correlations by mixing more and less recent experience for the updates, and rare experience will be used for more than just a single update. This was demonstrated in the Deep Q-Network (DQN) algorithm (Mnih et al., 2013; 2015), which stabilized the training of a value function, represented by a deep neural network, by using experience replay. Specifically, DQN used a large sliding window replay memory, sampled from it uniformly at random, and revisited each transition¹ eight times on average. In general, experience replay can reduce the amount of experience required to learn, and replace it with more computation and more memory – which are often cheaper resources than the RL agent’s interactions with its environment.

In this paper, we investigate how *prioritizing* which transitions are replayed can make experience replay more efficient and effective than if all transitions are replayed uniformly. The key idea is that an RL agent can learn more effectively from some transitions than from others. Transitions may be more or less surprising, redundant, or task-relevant. Some transitions may not be immediately useful to the agent, but might become so when the agent competence increases (Schmidhuber, 1991). Experience replay liberates online learning agents from processing transitions in the exact order they are experienced. Prioritized replay further liberates agents from considering transitions with the same frequency that they are experienced.

In particular, we propose to more frequently replay transitions with high expected learning progress, as measured by the magnitude of their temporal-difference (TD) error. This prioritization can lead to a loss of diversity, which we alleviate with stochastic prioritization, and introduce bias, which we correct with importance sampling. Our resulting algorithms are robust and scalable, which we demonstrate on the Atari 2600 benchmark suite, where we obtain faster learning and state-of-the-art performance.

¹ A transition is the atomic unit of interaction in RL, in our case a tuple of (state S_{t-1} , action A_{t-1} , reward R_t , discount γ_t , next state S_t). We choose this for simplicity, but most of the arguments in this paper also hold for a coarser ways of chunking experience, e.g. into sequences or episodes.

优先体验重播

Tom Schaul、John Quan、Ioannis Antonoglou 和 David Silver Google
DeepMind {schaul,johnquan,ioannisa,davidsilver}@google.com

抽象的

经验回放让在线强化学习代理能够记住并重用过去的经验。在之前的工作中，经验转换是从重播内存中统一采样的。然而，这种方法只是以最初经历的同频率重放转换，而不管它们的重要性如何。在本文中，我们开发了一个优先考虑经验的框架，以便更频繁地重播重要的转变，从而更有效地学习。我们在 Deep Q-Networks (DQN) 中使用优先体验回放，这是一种强化学习算法，在许多 Atari 游戏中实现了人类水平的性能。具有优先体验重播功能的 DQN 达到了新的最先进水平，优于 DQN，在 49 场比赛中的 41 场比赛中实现了统一重播。

1 简介

在线强化学习 (RL) 代理在观察经验流的同时逐步更新其参数 (策略、价值函数或模型)。以最简单的形式，它们在一次更新后立即丢弃传入的数据。与此相关的两个问题是 (a) 强相关更新会破坏独立同分布。许多流行的基于随机梯度的算法的假设，以及 (b) 快速忘记可能罕见的经验，这些经验以后会很有用。

Experience replay (Lin, 1992) 解决了这两个问题：通过将经验存储在重放存储器中，可以通过混合更多和更少的最近经验进行更新来打破时间相关性，并且罕见的经验将不仅仅用于单个更新。深度 Q 网络 (DQN) 算法 (Mnih 等人, 2013 年; 2015 年) 证明了这一点，该算法通过使用经验回放来稳定由深度神经网络表示的价值函数的训练。具体来说，DQN 使用大型滑动窗口重播内存，从中均匀随机采样，并平均重新访问每个转换¹ 八次。一般来说，经验回放可以减少学习所需的经验量，并用更多的计算和更多的内存取而代之——这通常是比 RL 代理与其环境交互更便宜的资源。

在本文中，我们研究了重放哪些转换的 *prioritizing* 如何使经验重放比统一重放所有转换更加高效和有效。关键思想是，强化学习代理可以从某些转换中比其他转换中更有效地学习。转变可能或多或少令人惊讶、多余或与任务相关。有些转换可能不会立即对代理有用，但当代理能力提高时可能会变得有用 (Schmidhuber, 1991)。经验回放将在线学习代理从按照其经历的确切顺序处理转换中解放出来。优先重放进一步使代理无需考虑与他们经历的频率相同的转换。

特别是，我们建议更频繁地重播具有较高预期学习进度的转换，这通过时间差异 (TD) 误差的大小来衡量。这种优先级可能会导致多样性的损失，我们可以通过随机优先级来缓解这种损失，并引入偏差，我们可以通过重要性抽样来纠正这种偏差。我们最终的算法是稳健且可扩展的，我们在 Atari 2600 基准套件上进行了演示，在那里我们获得了更快的学习速度和最先进的性能。

¹ A transition is the atomic unit of interaction in RL, in our case a tuple of (state S_{t-1} , action A_{t-1} , reward R_t , discount γ_t , next state S_t). We choose this for simplicity, but most of the arguments in this paper also hold for a coarser ways of chunking experience, e.g. into sequences or episodes.

2 BACKGROUND

Numerous neuroscience studies have identified evidence of experience replay in the hippocampus of rodents, suggesting that sequences of prior experience are replayed, either during awake resting or sleep. Sequences associated with rewards appear to be replayed more frequently (Atherton et al., 2015; Ólafsdóttir et al., 2015; Foster & Wilson, 2006). Experiences with high magnitude TD error also appear to be replayed more often (Singer & Frank, 2009; McNamara et al., 2014).

It is well-known that planning algorithms such as value iteration can be made more efficient by prioritizing updates in an appropriate order. *Prioritized sweeping* (Moore & Atkeson, 1993; Andre et al., 1998) selects which state to update next, prioritized according to the change in value, if that update was executed. The TD error provides one way to measure these priorities (van Seijen & Sutton, 2013). Our approach uses a similar prioritization method, but for model-free RL rather than model-based planning. Furthermore, we use a stochastic prioritization that is more robust when learning a function approximator from samples.

TD-errors have also been used as a prioritization mechanism for determining where to focus resources, for example when choosing where to explore (White et al., 2014) or which features to select (Geramifard et al., 2011; Sun et al., 2011).

In supervised learning, there are numerous techniques to deal with imbalanced datasets when class identities are known, including re-sampling, under-sampling and over-sampling techniques, possibly combined with ensemble methods (for a review, see Galar et al., 2012). A recent paper introduced a form of re-sampling in the context of deep RL with experience replay (Narasimhan et al., 2015); the method separates experience into two buckets, one for positive and one for negative rewards, and then picks a fixed fraction from each to replay. This is only applicable in domains that (unlike ours) have a natural notion of ‘positive/negative’ experience. Furthermore, Hinton (2007) introduced a form of non-uniform sampling based on error, with an importance sampling correction, which led to a 3x speed-up on MNIST digit classification.

There have been several proposed methods for playing Atari with deep reinforcement learning, including deep Q-networks (DQN) (Mnih et al., 2013; 2015; Guo et al., 2014; Stadie et al., 2015; Nair et al., 2015; Bellemare et al., 2016), and the *Double DQN* algorithm (van Hasselt et al., 2016), which is the current published state-of-the-art. Simultaneously with our work, an architectural innovation that separates advantages from the value function (see the co-submission by Wang et al., 2015) has also led to substantial improvements on the Atari benchmark.

3 PRIORITIZED REPLAY

Using a replay memory leads to design choices at two levels: which experiences to store, and which experiences to replay (and how to do so). This paper addresses only the latter: making the most effective use of the replay memory for learning, assuming that its contents are outside of our control (but see also Section 6).

3.1 A MOTIVATING EXAMPLE

To understand the potential gains of prioritization, we introduce an artificial ‘Blind Cliffwalk’ environment (described in Figure 1, left) that exemplifies the challenge of exploration when rewards are rare. With only n states, the environment requires an exponential number of random steps until the first non-zero reward; to be precise, the chance that a random sequence of actions will lead to the reward is 2^{-n} . Furthermore, the most relevant transitions (from rare successes) are hidden in a mass of highly redundant failure cases (similar to a bipedal robot falling over repeatedly, before it discovers how to walk).

We use this example to highlight the difference between the learning times of two agents. Both agents perform Q-learning updates on transitions drawn from the same replay memory. The first agent replays transitions uniformly at random, while the second agent invokes an oracle to prioritize transitions. This oracle greedily selects the transition that maximally reduces the global loss in its current state (in hindsight, after the parameter update). For the details of the setup, see Appendix B.1. Figure 1 (right) shows that picking the transitions in a good order can lead to exponential speed-ups

2 背景

许多神经科学研究已经确定了啮齿类动物海马体中经验重放的证据，表明先前的经验序列会在清醒休息或睡眠期间重放。与奖励相关的序列似乎会更频繁地重播（Atherton et al., 2015; Ólafsdóttir et al., 2015; Foster & Wilson, 2006）。具有高幅度 TD 误差的经验似乎也会更频繁地重播（Singer & Frank, 2009; McNamara 等人, 2014）。

众所周知，通过按适当的顺序确定更新的优先级，可以使诸如值迭代之类的规划算法变得更加高效。*Prioritized sweeping*（摩尔和阿特克森, 1993; Andre 等人, 1998）选择接下来要更新的状态，如果执行了更新，则根据值的变化确定优先级。TD 误差提供了一种衡量这些优先级的方法（van Seijen & Sutton, 2013）。我们的方法使用类似的优先级划分方法，但用于无模型强化学习而不是基于模型的规划。此外，我们使用随机优先级，在从样本中学习函数逼近器时，该优先级更加稳健。

TD 错误也被用作确定资源集中位置的优先级机制，例如在选择探索位置时（White et al., 2014）或选择哪些特征（Geramifard et al., 2011; Sun et al., 2011）。

在监督学习中，当类别身份已知时，有许多技术可以处理不平衡数据集，包括重采样、欠采样和过采样技术，可能与集成方法相结合（有关评论，请参阅 Galar 等人, 2012）。最近的一篇论文介绍了一种在深度强化学习背景下进行经验回放的重采样形式（Narasimhan et al., 2015）；该方法将经验分为两类，一类用于积极奖励，一类用于消极奖励，然后从每个桶中选择一个固定分数进行重播。这仅适用于（与我们不同的）具有“积极/消极”体验自然概念的领域。此外，Hinton（2007）引入了一种基于误差的非均匀采样形式，并进行了重要性采样校正，这使得 MNIST 数字分类速度提高了 3 倍。

已经提出了几种利用深度强化学习玩 Atari 的方法，包括深度 Q 网络（DQN）（Mnih 等人, 2013; 2015; Guo 等人, 2014; Stadie 等人, 2015; Nair 等人, 2015; Bellemare 等人, 2016）和 *Double DQN* 算法（van Hasselt 等人, 2016），这是当前发布的最新技术。在我们工作的同时，将优势与价值函数分开的架构创新（参见 Wang 等人, 2015 年共同提交的文章）也带来了 Atari 基准的实质性改进。

3 优先重播

使用重播内存会导致两个层面的设计选择：存储哪些体验，以及重播哪些体验（以及如何重播）。本文仅讨论后者：假设重放内存的内容超出了我们的控制范围，最有效地利用重放内存进行学习（但另请参阅第 6 节）。

3.1 一个激励人心的例子

为了理解优先顺序的潜在收益，我们引入了一个人工的“盲崖行走”环境（如图 1 左图所示），它举例说明了当奖励稀少时探索的挑战。只有 n 状态，环境需要指数数量的随机步骤，直到第一个非零奖励；准确地说，随机的一系列行动导致奖励的机会是 2^{-n} 。此外，最相关的转变（来自罕见的成功）隐藏在大量高度冗余的失败案例中（类似于双足机器人在发现如何行走之前反复摔倒）。

我们用这个例子来强调两个代理的学习时间之间的差异。两个代理对从同一重放内存中提取的转换执行 Q 学习更新。第一个代理随机地统一重播转换，而第二个代理调用预言机来确定转换的优先级。这个预言机贪婪地选择最大程度地减少当前状态下全局损失的转换（事后看来，在参数更新之后）。有关设置的详细信息，请参见附录 B.1。图 1（右）显示，以良好的顺序选择转换可以带来指数级的加速

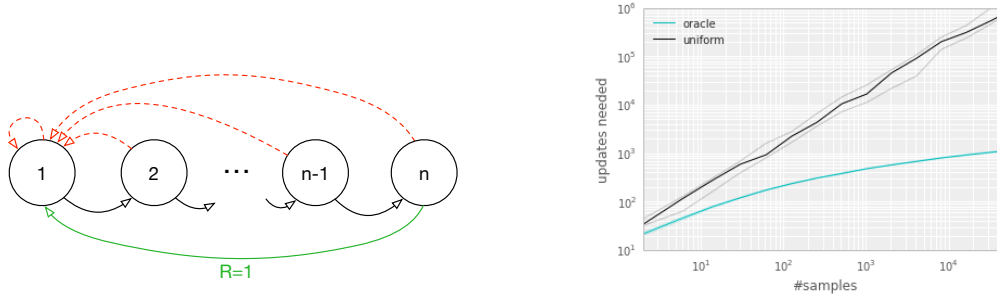


Figure 1: **Left:** Illustration of the ‘Blind Cliffwalk’ example domain: there are two actions, a ‘right’ and a ‘wrong’ one, and the episode is terminated whenever the agent takes the ‘wrong’ action (dashed red arrows). Taking the ‘right’ action progresses through a sequence of n states (black arrows), at the end of which lies a final reward of 1 (green arrow); reward is 0 elsewhere. We chose a representation such that generalizing over what action is ‘right’ is not possible. **Right:** Median number of learning steps required to learn the value function as a function of the size of the total number of transitions in the replay memory. Note the log-log scale, which highlights the exponential speed-up from replaying with an oracle (bright blue), compared to uniform replay (black); faint lines are min/max values from 10 independent runs.

over uniform choice. Such an oracle is of course not realistic, yet the large gap motivates our search for a practical approach that improves on uniform random replay.

3.2 PRIORITIZING WITH TD-ERROR

The central component of prioritized replay is the criterion by which the importance of each transition is measured. One idealised criterion would be the amount the RL agent can learn from a transition in its current state (expected learning progress). While this measure is not directly accessible, a reasonable proxy is the magnitude of a transition’s TD error δ , which indicates how ‘surprising’ or unexpected the transition is: specifically, how far the value is from its next-step bootstrap estimate (Andre et al., 1998). This is particularly suitable for incremental, online RL algorithms, such as SARSA or Q-learning, that already compute the TD-error and update the parameters in proportion to δ . The TD-error can be a poor estimate in some circumstances as well, e.g. when rewards are noisy; see Appendix A for a discussion of alternatives.

To demonstrate the potential effectiveness of prioritizing replay by TD error, we compare the uniform and oracle baselines in the Blind Cliffwalk to a ‘greedy TD-error prioritization’ algorithm. This algorithm stores the last encountered TD error along with each transition in the replay memory. The transition with the largest absolute TD error is replayed from the memory. A Q-learning update is applied to this transition, which updates the weights in proportion to the TD error. New transitions arrive without a known TD-error, so we put them at maximal priority in order to guarantee that all experience is seen at least once. Figure 2 (left), shows that this algorithm results in a substantial reduction in the effort required to solve the Blind Cliffwalk task.²

Implementation: To scale to large memory sizes N , we use a binary heap data structure for the priority queue, for which finding the maximum priority transition when sampling is $O(1)$ and updating priorities (with the new TD-error after a learning step) is $O(\log N)$. See Appendix B.2.1 for more details.

3.3 STOCHASTIC PRIORITIZATION

However, greedy TD-error prioritization has several issues. First, to avoid expensive sweeps over the entire replay memory, TD errors are only updated for the transitions that are replayed. One consequence is that transitions that have a low TD error on first visit may not be replayed for a long time (which means effectively never with a sliding window replay memory). Further, it is sensitive to noise spikes (e.g. when rewards are stochastic), which can be exacerbated by bootstrapping, where

² Note that a random (or optimistic) initialization of the Q-values is necessary with greedy prioritization. If initializing with zero instead, unrewarded transitions would appear to have zero error initially, be placed at the bottom of the queue, and not be revisited until the error on other transitions drops below numerical precision.

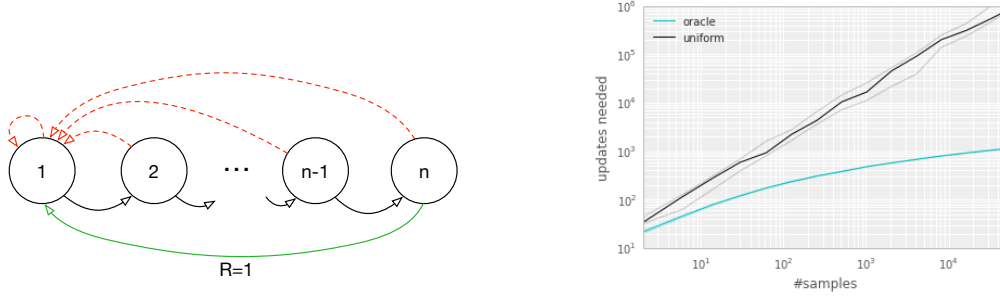


图 1: 左: “Blind Cliffwalk” 示例域的插图: 有两个操作, 一个“正确”和一个“错误”操作, 每当智能体采取“错误”操作 (红色虚线箭头) 时, 该情节就会终止。采取“正确”的行动会经历一系列 n 状态 (黑色箭头), 最后的奖励为 1 (绿色箭头); 其他地方奖励为 0。我们选择了一种表示方式, 以便概括什么行动是“正确的”是不可能的。右: 学习价值函数所需的学习步骤中位数, 作为重放存储器中转换总数大小的函数。请注意对数刻度, 它突出显示了与均匀重放 (黑色) 相比, 使用预言机重放 (亮蓝色) 带来的指数加速; 淡线是 10 次独立运行的最小/最大值。

超过统一的选择。这样的预言当然是不现实的, 但巨大的差距促使我们寻找一种改进均匀随机重放的实用方法。

3.2 TD 误差的优先级

优先重放的核心组成部分是衡量每个转换重要性的标准。一个理想化的标准是强化学习代理可以从当前状态的转换中学习的数量 (预期学习进度)。虽然这个衡量标准无法直接获得, 但一个合理的代理是转变的 TD 误差 δ 的大小, 它表明转变的“令人惊讶”或意外程度: 具体来说, 该值与其下一步引导估计值相差多远 (Andre 等人, 1998)。这特别适合增量在线 RL 算法, 例如 SARSA 或 Q-learning, 它们已经计算了 TD 误差并按 δ 的比例更新参数。在某些情况下, TD 误差也可能是一个糟糕的估计, 例如当奖励嘈杂时; 有关替代方案的讨论, 请参阅附录 A。

为了证明通过 TD 错误对重放进行优先级排序的潜在有效性, 我们将 Blind Cliffwalk 中的统一基线和预言机基线与“贪婪 TD 错误优先级”算法进行了比较。该算法将最后遇到的 TD 错误与每个转换一起存储在重放存储器中。从内存中重放具有最大绝对 TD 误差的转变。Q-learning 更新应用于此转换, 根据 TD 误差按比例更新权重。新的转换到达时没有已知的 TD 错误, 因此我们将它们置于最高优先级, 以保证所有体验至少被看到一次。图 2 (左) 显示该算法大大减少了解决 Blind Cliffwalk 任务所需的工作量。²

实现: 为了扩展到大内存大小 N , 我们对优先级队列使用二进制堆数据结构, 在采样时找到最大优先级转换为 $O(1)$, 更新优先级 (在学习步骤后使用新的 TD 误差) 为 $O(\log N)$ 。更多详细信息, 请参阅附录 B.2.1。

3.3 随机优先级

然而, 贪婪的 TD 错误优先级有几个问题。首先, 为了避免对整个重放存储器进行昂贵的扫描, 仅针对重放的转换更新 TD 错误。一个结果是, 第一次访问时具有低 TD 错误的转换可能不会在很长一段时期内重放 (这意味着实际上永远不会使用滑动窗口重放内存)。此外, 它对噪声尖峰很敏感 (例如, 当奖励是随机的时), 这种情况可能会因引导而加剧, 其中

² Note that a random (or optimistic) initialization of the Q-values is necessary with greedy prioritization. If initializing with zero instead, unrewarded transitions would appear to have zero error initially, be placed at the bottom of the queue, and not be revisited until the error on other transitions drops below numerical precision.

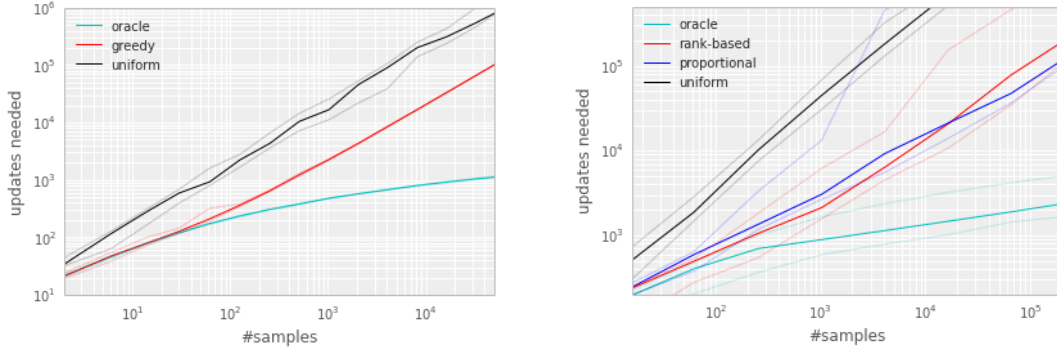


Figure 2: Median number of updates required for Q-learning to learn the value function on the Blind Cliffwalk example, as a function of the total number of transitions (only a single one of which was successful and saw the non-zero reward). Faint lines are min/max values from 10 random initializations. Black is uniform random replay, cyan uses the hindsight-oracle to select transitions, red and blue use prioritized replay (rank-based and proportional respectively). The results differ by multiple orders of magnitude, thus the need for a log-log plot. In both subplots it is evident that replaying experience in the right order makes an enormous difference to the number of updates required. See Appendix B.1 for details. **Left:** Tabular representation, greedy prioritization. **Right:** Linear function approximation, both variants of stochastic prioritization.

approximation errors appear as another source of noise. Finally, greedy prioritization focuses on a small subset of the experience: errors shrink slowly, especially when using function approximation, meaning that the initially high error transitions get replayed frequently. This lack of diversity that makes the system prone to over-fitting.

To overcome these issues, we introduce a stochastic sampling method that interpolates between pure greedy prioritization and uniform random sampling. We ensure that the probability of being sampled is monotonic in a transition’s priority, while guaranteeing a non-zero probability even for the lowest-priority transition. Concretely, we define the probability of sampling transition i as

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (1)$$

where $p_i > 0$ is the priority of transition i . The exponent α determines how much prioritization is used, with $\alpha = 0$ corresponding to the uniform case.

The first variant we consider is the direct, proportional prioritization where $p_i = |\delta_i| + \epsilon$, where ϵ is a small positive constant that prevents the edge-case of transitions not being revisited once their error is zero. The second variant is an indirect, rank-based prioritization where $p_i = \frac{1}{\text{rank}(i)}$, where $\text{rank}(i)$ is the rank of transition i when the replay memory is sorted according to $|\delta_i|$. In this case, P becomes a power-law distribution with exponent α . Both distributions are monotonic in $|\delta|$, but the latter is likely to be more robust, as it is insensitive to outliers. Both variants of stochastic prioritization lead to large speed-ups over the uniform baseline on the Cliffwalk task, as shown on Figure 2 (right).

Implementation: To efficiently sample from distribution (1), the complexity cannot depend on N . For the rank-based variant, we can approximate the cumulative density function with a piecewise linear function with k segments of equal probability. The segment boundaries can be precomputed (they change only when N or α change). At runtime, we sample a segment, and then sample uniformly among the transitions within it. This works particularly well in conjunction with a minibatch-based learning algorithm: choose k to be the size of the minibatch, and sample exactly one transition from each segment – this is a form of stratified sampling that has the added advantage of balancing out the minibatch (there will always be exactly one transition with high magnitude δ , one with medium magnitude, etc). The proportional variant is different, also admits an efficient implementation based on a ‘sum-tree’ data structure (where every node is the sum of its children, with the priorities as the leaf nodes), which can be efficiently updated and sampled from. See Appendix B.2.1 for more additional details.

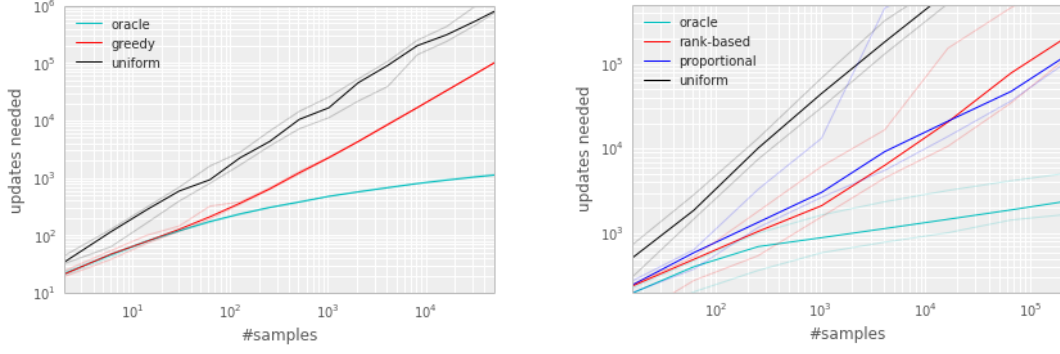


图 2: Q-learning 学习 Blind Cliffwalk 示例中的价值函数所需的更新中位数，作为转换总数的函数（其中只有一个转换成功并获得非零奖励）。淡线是 10 次随机初始化的最小/最大值。黑色是统一随机重放，青色使用事后预言来选择转换，红色和蓝色使用优先重放（分别基于排名和比例）。结果相差多个数量级，因此需要双对数图。在这两个子图中，很明显，以正确的顺序重播体验会对所需的更新数量产生巨大的影响。详细信息请参见附录 B.1。左：表格表示，贪婪优先级。右：线性函数近似，随机优先级的两种变体。

近似误差是另一个噪声源。最后，贪婪优先级侧重于体验的一小部分：错误缩小缓慢，尤其是在使用函数逼近时，这意味着最初的高错误转换会频繁重播。这种多样性的缺乏使得系统容易出现过度拟合。

为了克服这些问题，我们引入了一种随机采样方法，该方法在纯贪婪优先级和均匀随机采样之间进行插值。我们确保被采样的概率在转换的优先级中是单调的，同时保证即使对于最低优先级的转换也有非零概率。具体来说，我们将采样转移 i 的概率定义为

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (1)$$

其中 $p_i > 0$ 是转换 i 的优先级。指数 α 确定使用多少优先级，其中 $\alpha = 0$ 对应于统一情况。

我们考虑的第一个变体是直接的比例优先级，其中 $p_i = |\delta_i| + \epsilon$ ，其中 ϵ 是一个小的正常数，可以防止一旦误差为零，就不会重新访问过渡的边缘情况。第二种变体是间接的、基于等级的优先级排序，其中 $p_i = \frac{1}{\text{rank}(i)}$ ，其中， $\text{rank}(i)$ 是当重放内存根据 $|\delta_i|$ 排序时转换 i 的等级。在这种情况下， P 成为指数为 α 的幂律分布。两种分布在 $|\delta|$ 中都是单调的，但后者可能更稳健，因为它对异常值不敏感。随机优先级的两种变体都会导致 Cliffwalk 任务在统一基线上大幅加速，如图 2（右）所示。

实现：为了有效地从分布 (1) 中采样，复杂性不能依赖于 N 。对于基于排序的变体，我们可以使用具有等概率的 k 段的分段线性函数来近似累积密度函数。段边界可以预先计算（它们仅在 N 或 α 更改时更改）。在运行时，我们对一个片段进行采样，然后在其中的转换之间进行均匀采样。这与基于小批量的学习算法结合起来效果特别好：选择 k 作为小批量的大小，并从每个片段中精确采样一个转换 - 这是一种分层采样的形式，具有平衡小批量的额外优势（总是有一个高幅度的转换 δ ，一个中等幅度的转换，等等）。比例变体是不同的，它也允许基于“和树”数据结构的有效实现（其中每个节点都是其子节点的总和，优先级作为叶节点），可以有效地更新和采样。更多详细信息请参见附录 B.2.1。

Algorithm 1 Double DQN with proportional prioritization

```

1: Input: minibatch  $k$ , step-size  $\eta$ , replay period  $K$  and size  $N$ , exponents  $\alpha$  and  $\beta$ , budget  $T$ .
2: Initialize replay memory  $\mathcal{H} = \emptyset$ ,  $\Delta = 0$ ,  $p_1 = 1$ 
3: Observe  $S_0$  and choose  $A_0 \sim \pi_\theta(S_0)$ 
4: for  $t = 1$  to  $T$  do
5:   Observe  $S_t, R_t, \gamma_t$ 
6:   Store transition  $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$  in  $\mathcal{H}$  with maximal priority  $p_t = \max_{i < t} p_i$ 
7:   if  $t \equiv 0 \pmod K$  then
8:     for  $j = 1$  to  $k$  do
9:       Sample transition  $j \sim P(j) = p_j^\alpha / \sum_i p_i^\alpha$ 
10:      Compute importance-sampling weight  $w_j = (N \cdot P(j))^{-\beta} / \max_i w_i$ 
11:      Compute TD-error  $\delta_j = R_j + \gamma_j Q_{\text{target}}(S_j, \arg \max_a Q(S_j, a)) - Q(S_{j-1}, A_{j-1})$ 
12:      Update transition priority  $p_j \leftarrow |\delta_j|$ 
13:      Accumulate weight-change  $\Delta \leftarrow \Delta + w_j \cdot \delta_j \cdot \nabla_\theta Q(S_{j-1}, A_{j-1})$ 
14:    end for
15:    Update weights  $\theta \leftarrow \theta + \eta \cdot \Delta$ , reset  $\Delta = 0$ 
16:    From time to time copy weights into target network  $\theta_{\text{target}} \leftarrow \theta$ 
17:  end if
18:  Choose action  $A_t \sim \pi_\theta(S_t)$ 
19: end for

```

3.4 ANNEALING THE BIAS

The estimation of the expected value with stochastic updates relies on those updates corresponding to the same distribution as its expectation. Prioritized replay introduces bias because it changes this distribution in an uncontrolled fashion, and therefore changes the solution that the estimates will converge to (even if the policy and state distribution are fixed). We can correct this bias by using importance-sampling (IS) weights

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta$$

that fully compensates for the non-uniform probabilities $P(i)$ if $\beta = 1$. These weights can be folded into the Q-learning update by using $w_i \delta_i$ instead of δ_i (this is thus *weighted* IS, not ordinary IS, see e.g. Mahmood et al., 2014). For stability reasons, we always normalize weights by $1 / \max_i w_i$ so that they only scale the update downwards.

In typical reinforcement learning scenarios, the unbiased nature of the updates is most important near convergence at the end of training, as the process is highly non-stationary anyway, due to changing policies, state distributions and bootstrap targets; we hypothesize that a small bias can be ignored in this context (see also Figure 12 in the appendix for a case study of full IS correction on Atari). We therefore exploit the flexibility of *annealing* the amount of importance-sampling correction over time, by defining a schedule on the exponent β that reaches 1 only at the end of learning. In practice, we linearly anneal β from its initial value β_0 to 1. Note that the choice of this hyperparameter interacts with choice of prioritization exponent α ; increasing both simultaneously prioritizes sampling more aggressively at the same time as correcting for it more strongly.

Importance sampling has another benefit when combined with prioritized replay in the context of non-linear function approximation (e.g. deep neural networks): here large steps can be very disruptive, because the first-order approximation of the gradient is only reliable locally, and have to be prevented with a smaller global step-size. In our approach instead, prioritization makes sure high-error transitions are seen many times, while the IS correction reduces the gradient magnitudes (and thus the effective step size in parameter space), and allowing the algorithm follow the curvature of highly non-linear optimization landscapes because the Taylor expansion is constantly re-approximated.

We combine our prioritized replay algorithm into a full-scale reinforcement learning agent, based on the state-of-the-art Double DQN algorithm. Our principal modification is to replace the uniform random sampling used by Double DQN with our stochastic prioritization and importance sampling methods (see Algorithm 1).

Algorithm 1 Double DQN with proportional prioritization

```

1: Input: minibatch  $k$ , step-size  $\eta$ , replay period  $K$  and size  $N$ , exponents  $\alpha$  and  $\beta$ , budget  $T$ .
2: Initialize replay memory  $\mathcal{H} = \emptyset$ ,  $\Delta = 0$ ,  $p_1 = 1$ 
3: Observe  $S_0$  and choose  $A_0 \sim \pi_\theta(S_0)$ 
4: for  $t = 1$  to  $T$  do
5:   Observe  $S_t, R_t, \gamma_t$ 
6:   Store transition  $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$  in  $\mathcal{H}$  with maximal priority  $p_t = \max_{i < t} p_i$ 
7:   if  $t \equiv 0 \pmod K$  then
8:     for  $j = 1$  to  $k$  do
9:       Sample transition  $j \sim P(j) = p_j^\alpha / \sum_i p_i^\alpha$ 
10:      Compute importance-sampling weight  $w_j = (N \cdot P(j))^{-\beta} / \max_i w_i$ 
11:      Compute TD-error  $\delta_j = R_j + \gamma_j Q_{\text{target}}(S_j, \arg \max_a Q(S_j, a)) - Q(S_{j-1}, A_{j-1})$ 
12:      Update transition priority  $p_j \leftarrow |\delta_j|$ 
13:      Accumulate weight-change  $\Delta \leftarrow \Delta + w_j \cdot \delta_j \cdot \nabla_\theta Q(S_{j-1}, A_{j-1})$ 
14:    end for
15:    Update weights  $\theta \leftarrow \theta + \eta \cdot \Delta$ , reset  $\Delta = 0$ 
16:    From time to time copy weights into target network  $\theta_{\text{target}} \leftarrow \theta$ 
17:  end if
18:  Choose action  $A_t \sim \pi_\theta(S_t)$ 
19: end for

```

3.4 偏差退火

使用随机更新对期望值的估计依赖于与其期望相同的分布相对应的那些更新。优先重放引入了偏差，因为它以不受控制的方式改变了这种分布，从而改变了估计将收敛到的解决方案（即使策略和状态分布是固定的）。我们可以通过使用重要性采样（IS）权重来纠正这种偏差

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta$$

如果 $\beta = 1$ ，则完全补偿非均匀概率 $P(i)$ 。可以通过使用 $w_i \delta_i$ 而不是 δ_i （将这些权重折叠到 Q 学习更新中，因此这是 *weighted IS*，而不是普通的 IS，请参见例如马哈茂德等人，2014）。出于稳定性原因，我们始终将权重标准化为 $1 / \max_i w_i$ ，以便它们只会向下缩放更新。

在典型的强化学习场景中，更新的无偏性在训练结束时接近收敛时最为重要，因为由于政策、状态分布和引导目标的变化，该过程无论如何都是高度非平稳的；我们假设在这种情况下可以忽略一个小偏差（另请参见附录中的图 12，了解 Atari 上完整 IS 校正的案例研究）。因此，我们通过在指数 β 上定义一个时间表，仅在学习结束时达到 1，来利用 *annealing* 随时间的重要性采样校正量的灵活性。在实践中，我们将 β 从其初始值 β_0 线性退火到 1。请注意，该超参数的选择与优先级指数 α 的选择相互作用；同时增加两者可以优先考虑更积极地采样，同时更强烈地纠正采样。

当在非线性函数逼近（例如深度神经网络）的背景下与优先重放相结合时，重要性采样还有另一个好处：这里大的步长可能非常具有破坏性，因为梯度的一阶逼近仅在局部可靠，并且必须用较小的全局步长来防止。相反，在我们的方法中，优先级确保多次出现高误差转换，而 IS 校正减少了梯度幅度（从而减少了参数空间中的有效步长），并允许算法遵循高度非线性优化景观的曲率，因为泰勒展开不断地重新逼近。

我们基于最先进的 Double DQN 算法，将优先重放算法结合到全面的强化学习代理中。我们的主要修改是用我们的随机优先级和重要性采样方法替换 Double DQN 使用的均匀随机采样（参见算法 1）。

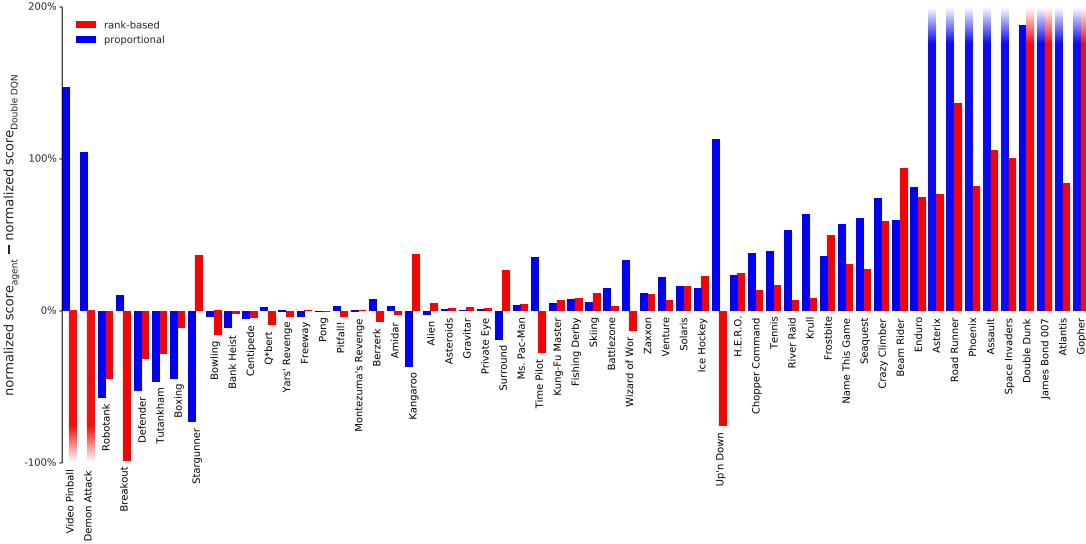


Figure 3: Difference in normalized score (the gap between random and human is 100%) on 57 games with human starts, comparing Double DQN with and without prioritized replay (rank-based variant in red, proportional in blue), showing substantial improvements in most games. Exact scores are in Table 6. See also Figure 9 where regular DQN is the baseline.

4 ATARI EXPERIMENTS

With all these concepts in place, we now investigate to what extent replay with such prioritized sampling can improve performance in realistic problem domains. For this, we chose the collection of Atari benchmarks (Bellemare et al., 2012) with their end-to-end RL from vision setup, because they are popular and contain diverse sets of challenges, including delayed credit assignment, partial observability, and difficult function approximation (Mnih et al., 2015; van Hasselt et al., 2016). Our hypothesis is that prioritized replay is generally useful, so that it will make learning with experience replay more efficient without requiring careful problem-specific hyperparameter tuning.

We consider two baseline algorithms that use uniform experience replay, namely the version of the DQN algorithm from the Nature paper (Mnih et al., 2015), and its recent extension Double DQN (van Hasselt et al., 2016) that substantially improved the state-of-the-art by reducing the over-estimation bias with Double Q-learning (van Hasselt, 2010). Throughout this paper we use the tuned version of the Double DQN algorithm. For this paper, the most relevant component of these baselines is the replay mechanism: all experienced transitions are stored in a sliding window memory that retains the last 10^6 transitions. The algorithm processes minibatches of 32 transitions sampled uniformly from the memory. One minibatch update is done for each 4 new transitions entering the memory, so all experience is replayed 8 times on average. Rewards and TD-errors are clipped to fall within $[-1, 1]$ for stability reasons.

We use the identical neural network architecture, learning algorithm, replay memory and evaluation setup as for the baselines (see Appendix B.2). The only difference is the mechanism for sampling transitions from the replay memory, with is now done according to Algorithm 1 instead of uniformly. We compare the baselines to both variants of prioritized replay (rank-based and proportional).

Only a single hyperparameter adjustment was necessary compared to the baseline: Given that prioritized replay picks high-error transitions more often, the typical gradient magnitudes are larger, so we reduced the step-size η by a factor 4 compared to the (Double) DQN setup. For the α and β_0 hyperparameters that are introduced by prioritization, we did a coarse grid search (evaluated on a subset of 8 games), and found the sweet spot to be $\alpha = 0.7$, $\beta_0 = 0.5$ for the rank-based variant and $\alpha = 0.6$, $\beta_0 = 0.4$ for the proportional variant. These choices are trading off aggressiveness with robustness, but it is easy to revert to a behavior closer to the baseline by reducing α and/or increasing β .

We produce the results by running each variant with a single hyperparameter setting across all games, as was done for the baselines. Our main evaluation metric is the *quality of the best policy*,

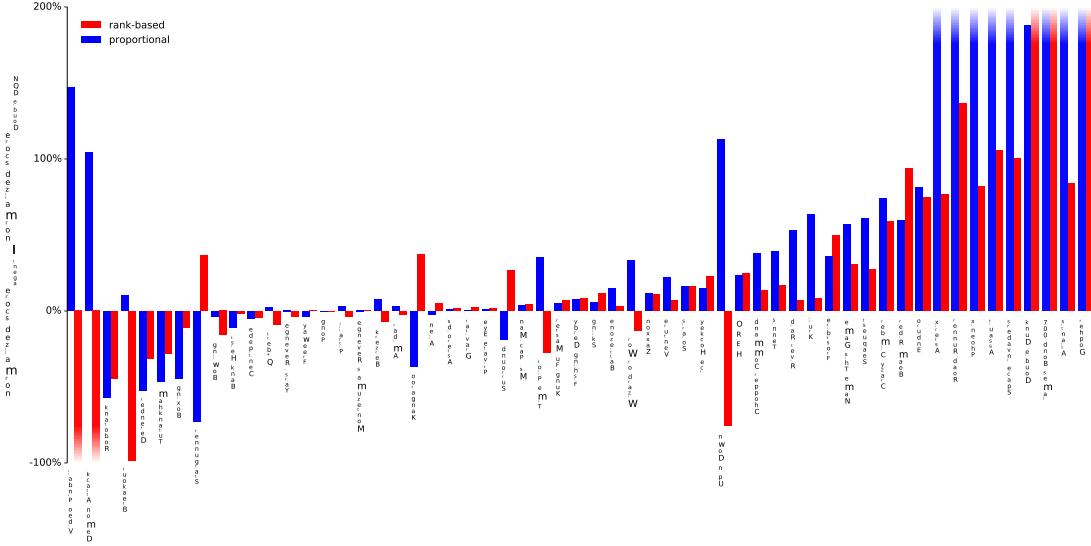


图 3: 在 57 场人类开始的游戏里, 归一化分数的差异 (随机分数与人类分数之间的差距为 100%), 比较有和没有优先重放的 Double DQN (红色为基于排名的变体, 蓝色为比例), 显示大多数游戏都有显著改进。确切的分数如表 6 所示。另请参见图 9, 其中常规 DQN 是基线。

4 雅达利实验

有了所有这些概念, 我们现在研究使用这种优先采样的重放可以在多大程度上提高实际问题领域的性能。为此, 我们选择了 Atari 基准集合 (Bellemare 等人, 2012) 以及来自视觉设置的端到端 RL, 因为它们很受欢迎并且包含不同的挑战, 包括延迟信用分配、部分可观察性和困难函数逼近 (Mnih 等人, 2015; van Hasselt 等人, 2016)。我们的假设是, 优先重放通常是有用的, 因此它将使经验重放的学习更加有效, 而不需要针对特定问题进行仔细的超参数调整。

我们考虑两种使用统一经验重放的基线算法, 即 Nature 论文中的 DQN 算法版本 (Mnih 等人, 2015 年) 及其最近的扩展 Double DQN (van Hasselt 等人, 2016 年), 该算法通过使用 Double Q 学习减少高估偏差, 大大提高了现有技术 (van Hasselt, 2010 年)。在本文中, 我们使用 Double DQN 算法的调整版本。对于本文来说, 这些基线最相关的组成部分是重播机制: 所有经历过的转换都存储在保留最后 10^6 转换的滑动窗口内存中。该算法处理从内存中均匀采样的 32 个转换的小批量。每 4 个新的转换进入内存就会进行一次小批量更新, 因此所有经验平均会重播 8 次。出于稳定性原因, 奖励和 TD 错误被限制在 $[-1, 1]$ 范围内。

我们使用与基线相同的神经网络架构、学习算法、重放内存和评估设置 (参见附录 B.2)。唯一的区别是从重放内存中采样转换的机制, 现在是根据算法 1 而不是统一完成。我们将基线与优先重放的两种变体 (基于排名和比例) 进行比较。

与基线相比, 只需要进行一次超参数调整: 考虑到优先重播更频繁地选择高错误转换, 典型的梯度幅度更大, 因此与 (双) DQN 设置相比, 我们将步长 η 减小了 4 倍。对于优先引入的 α 和 β_0 超参数, 我们进行了粗略网格搜索 (在 8 个游戏的子集上进行评估), 发现基于排名的变体的最佳位置为 $\alpha = 0.7$ 、 $\beta_0 = 0.5$, 对于比例变体的最佳位置为 $\alpha = 0.6$ 、 $\beta_0 = 0.4$ 。这些选择权衡了攻击性和稳健性, 但通过减少 α 和/或增加 β 很容易恢复到更接近基线的行为。

我们通过在所有游戏中使用单个超参数设置运行每个变体来生成结果, 就像对基线所做的那样。我们的主要评估指标是 *quality of the best policy*,

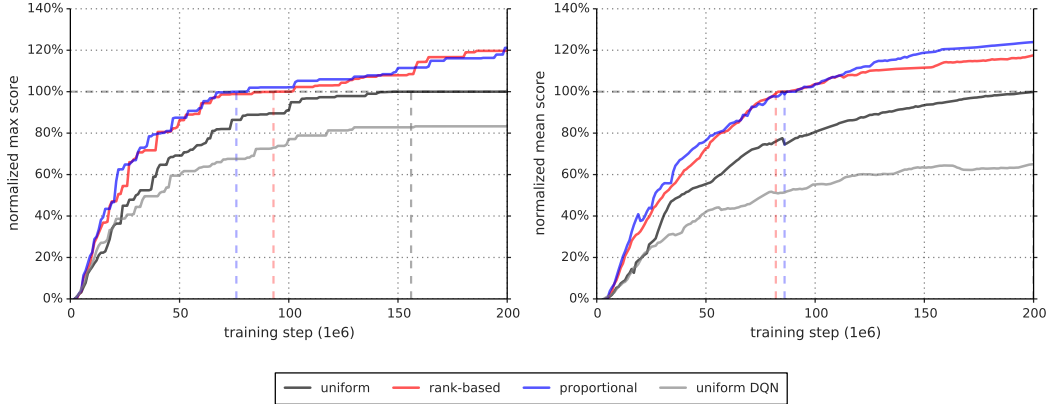


Figure 4: Summary plots of learning speed. **Left:** median over 57 games of the maximum baseline-normalized score achieved so far. The baseline-normalized score is calculated as in Equation 4 but using the maximum Double DQN score seen across training is used instead of the human score. The equivalence points are highlighted with dashed lines; those are the steps at which the curves reach 100%, (i.e., when the algorithm performs equivalently to Double DQN in terms of median over games). For rank-based and proportional prioritization these are at 47% and 38% of total training time. **Right:** Similar to the left, but using the mean instead of maximum, which captures cumulative performance rather than peak performance. Here rank-based and proportional prioritization reach the equivalence points at 41% and 43% of total training time, respectively. For the detailed learning curves that these plots summarize, see Figure 7.

in terms of average score per episode, given start states sampled from human traces (as introduced in Nair et al., 2015 and used in van Hasselt et al., 2016, which requires more robustness and generalization as the agent cannot rely on repeating a single memorized trajectory). These results are summarized in Table 1 and Figure 3, but full results and raw scores can be found in Tables 7 and 6 in the Appendix. A secondary metric is the *learning speed*, which we summarize on Figure 4, with more detailed learning curves on Figures 7 and 8.

	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Median	48%	106%	111%	113%	128%
Mean	122%	355%	418%	454%	551%
> baseline	–	41	–	38	42
> human	15	25	30	33	33
# games	49	49	57	57	57

Table 1: Summary of normalized scores. See Table 6 in the appendix for full results.

We find that adding prioritized replay to DQN leads to a substantial improvement in score on 41 out of 49 games (compare columns 2 and 3 of Table 6 or Figure 9 in the appendix), with the median normalized performance across 49 games increasing from 48% to 106%. Furthermore, we find that the boost from prioritized experience replay is *complementary* to the one from introducing Double Q-learning into DQN: performance increases another notch, leading to the current state-of-the-art on the Atari benchmark (see Figure 3). Compared to Double DQN, the median performance across 57 games increased from 111% to 128%, and the mean performance from 418% to 551% bringing additional games such as River Raid, Seaquest and Surround to a human level for the first time, and making large jumps on others (e.g. Gopher, James Bond 007 or Space Invaders). Note that mean performance is not a very reliable metric because a single game (Video Pinball) has a dominant contribution. Prioritizing replay gives a performance boost on almost all games, and on aggregate, learning is twice as fast (see Figures 4 and 8). The learning curves on Figure 7 illustrate that while the two variants of prioritization usually lead to similar results, there are games where one of them remains close to the Double DQN baseline while the other one leads to a big boost, for example Double Dunk or Surround for the rank-based variant, and Alien, Asterix, Enduro, Phoenix or Space Invaders for the proportional variant. Another observation from the learning curves is that compared to the uniform baseline, prioritization is effective at reducing the delay until performance gets off the ground in games that otherwise suffer from such a delay, such as Battlezone, Zaxxon or Frostbite.

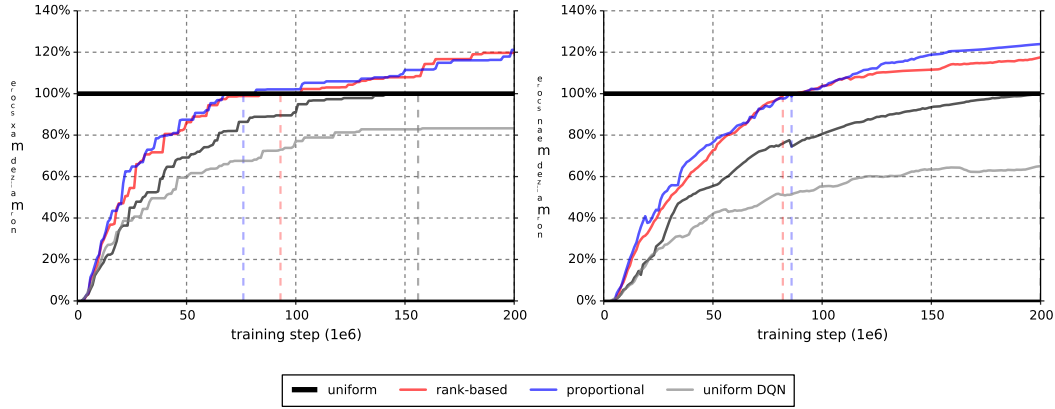


图 4: 学习速度的汇总图。左: 迄今为止 57 场比赛中达到的最大基线归一化得分的中位数。基线归一化分数的计算方式如公式 4 所示, 但使用训练过程中看到的最大 Double DQN 分数而不是人类分数。当量点用虚线突出显示; 这些是曲线达到 100% 的步骤 (即, 当算法在游戏中位数方面与 Double DQN 等效时)。对于基于排名和比例的优先级, 它们分别占总训练时间的 47% 和 38%。右图: 与左图类似, 但使用平均值而不是最大值, 捕获累积性能而不是峰值性能。这里, 基于排名的优先级和比例优先级分别在总训练时间的 41% 和 43% 处达到等价点。有关这些图总结的详细学习曲线, 请参见图 7。

就每集的平均得分而言, 给定从人类痕迹中采样的开始状态 (如 Nair 等人, 2015 年引入并在 van Hasselt 等人, 2016 年使用, 这需要更高的鲁棒性和泛化性, 因为智能体不能依赖于重复单个记忆轨迹)。表 1 和图 3 总结了这些结果, 但完整结果和原始分数可以在附录的表 7 和表 6 中找到。第二个指标是 *learning speed*, 我们在图 4 中对其进行了总结, 在图 7 和 8 中提供了更详细的学习曲线。

	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Median	48%	106%	111%	113%	128%
Mean	122%	355%	418%	454%	551%
> baseline	–	41	–	38	42
> human	15	25	30	33	33
# games	49	49	57	57	57

表 1: 归一化分数摘要。完整结果请参见附录中的表 6。

我们发现, 在 DQN 中添加优先重放可以显著提高 49 场游戏中 41 场的得分 (比较表 6 的第 2 列和第 3 列或附录中的图 9), 49 场游戏的中位标准化性能从 48% 增加到 106%。此外, 我们发现优先体验重放相对于将双 Q 学习引入 DQN 的提升是 *complementary*: 性能又提高了一个档次, 达到了 Atari 基准测试的当前最先进水平 (见图 3)。与 Double DQN 相比, 57 款游戏的中值性能从 111% 提高到 128%, 平均性能从 418% 提高到 551%, 使《River Raid》、《Seaquest》和《Surround》等其他游戏首次达到人类水平, 并大幅超越其他游戏 (例如《地鼠》、《詹姆斯·邦德 007》或《太空侵略者》)。请注意, 平均性能并不是一个非常可靠的指标, 因为单个游戏 (视频弹球) 具有主导贡献。优先考虑重播可以提高几乎所有游戏的性能, 总体而言, 学习速度提高了一倍 (见图 4 和图 8)。图 7 中的学习曲线表明, 虽然优先级的两种变体通常会产生相似的结果, 但在某些游戏中, 其中一种仍然接近 Double DQN 基线, 而另一种则导致大幅提升, 例如基于排名的变体的 Double Dunk 或 Surround, 以及比例变体的 Alien、Asterix、Enduro、Phoenix 或 Space Invaders。从学习曲线中观察到的另一个结果是, 与统一基线相比, 优先级可以有效地减少延迟, 直到游戏性能达到预期为止, 否则这些游戏就会出现延迟, 例如《Battlezone》、《Zaxxon》或《Frostbite》。

5 DISCUSSION

In the head-to-head comparison between rank-based prioritization and proportional prioritization, we expected the rank-based variant to be more robust because it is not affected by outliers nor error magnitudes. Furthermore, its heavy-tail property also guarantees that samples will be diverse, and the stratified sampling from partitions of different errors will keep the total minibatch gradient at a stable magnitude throughout training. On the other hand, the ranks make the algorithm blind to the relative error scales, which could incur a performance drop when there is structure in the distribution of errors to be exploited, such as in sparse reward scenarios. Perhaps surprisingly, both variants perform similarly in practice; we suspect this is due to the heavy use of clipping (of rewards and TD-errors) in the DQN algorithm, which removes outliers. Monitoring the distribution of TD-errors as a function of time for a number of games (see Figure 10 in the appendix), and found that it becomes close to a heavy-tailed distribution as learning progresses, while still differing substantially across games; this empirically validates the form of Equation 1. Figure 11, in the appendix, shows how this distribution interacts with Equation 1 to produce the effective replay probabilities.

While doing this analysis, we stumbled upon another phenomenon (obvious in retrospect), namely that some fraction of the visited transitions are never replayed before they drop out of the sliding window memory, and many more are replayed for the first time only long after they are encountered. Also, uniform sampling is implicitly biased toward out-of-date transitions that were generated by a policy that has typically seen hundreds of thousands of updates since. Prioritized replay with its bonus for unseen transitions directly corrects the first of these issues, and also tends to help with the second one, as more recent transitions tend to have larger error – this is because old transitions will have had more opportunities to have them corrected, and because novel data tends to be less well predicted by the value function.

We hypothesize that deep neural networks interact with prioritized replay in another interesting way. When we distinguish learning the value given a representation (i.e., the top layers) from learning an improved representation (i.e., the bottom layers), then transitions for which the representation is good will quickly reduce their error and then be replayed much less, increasing the learning focus on others where the representation is poor, thus putting more resources into distinguishing aliased states – if the observations and network capacity allow for it.

6 EXTENSIONS

Prioritized Supervised Learning: The analogous approach to prioritized replay in the context of supervised learning is to sample non-uniformly from the dataset, each sample using a priority based on its last-seen error. This can help focus the learning on those samples that can still be learned from, devoting additional resources to the (hard) boundary cases, somewhat similarly to boosting (Galar et al., 2012). Furthermore, if the dataset is imbalanced, we hypothesize that samples from the rare classes will be sampled disproportionately often, because their errors shrink less fast, and the chosen samples from the common classes will be those nearest to the decision boundaries, leading to an effect similar to hard negative mining (Felzenszwalb et al., 2008). To check whether these intuitions hold, we conducted a preliminary experiment on a class-imbalanced variant of the classical MNIST digit classification problem (LeCun et al., 1998), where we removed 99% of the samples for digits 0, 1, 2, 3, 4 in the training set, while leaving the test/validation sets untouched (i.e., those retain class balance). We compare two scenarios: in the informed case, we reweight the errors of the impoverished classes artificially (by a factor 100), in the uninformed scenario, we provide no hint that the test distribution will differ from the training distribution. See Appendix B.3 for the details of the convolutional neural network training setup. Prioritized sampling (uninformed, with $\alpha = 1$, $\beta = 0$) outperforms the uninformed uniform baseline, and approaches the performance of the informed uniform baseline in terms of generalization (see Figure 5); again, prioritized training is also faster in terms of learning speed.

Off-policy Replay: Two standard approaches to off-policy RL are rejection sampling and using importance sampling ratios ρ to correct for how likely a transition would have been on-policy. Our approach contains analogues to both these approaches, the replay probability P and the IS-correction w . It appears therefore natural to apply it to off-policy RL, if transitions are available in a replay memory. In particular, we recover weighted IS with $w = \rho$, $\alpha = 0$, $\beta = 1$ and rejection sampling

5 讨论

在基于排名的优先级和比例优先级之间的直接比较中，我们预计基于排名的变体更加稳健，因为它不受异常值或误差大小的影响。此外，它的重尾特性还保证了样本的多样性，并且从不同误差的分区中进行分层采样将使整个小批量梯度在整个训练过程中保持稳定的大小。另一方面，排名使算法对相对误差范围视而不见，当要利用的误差分布存在结构时（例如在稀疏奖励场景中），这可能会导致性能下降。也许令人惊讶的是，这两种变体在实践中表现相似。我们怀疑这是由于 DQN 算法中大量使用了裁剪（奖励和 TD 错误），从而消除了异常值。监控多个游戏中 TD 误差随时间的分布（参见附录中的图 10），发现随着学习的进展，它变得接近重尾分布，但不同游戏之间仍然存在很大差异；这从经验上验证了方程 1 的形式。附录中的图 11 显示了该分布如何与方程 1 相互作用以产生有效的重放概率。

在进行此分析时，我们偶然发现了另一个现象（回顾起来很明显），即访问过的转换的某些部分在退出滑动窗口内存之前从未重放，而更多的部分只有在遇到很长时间后才第一次重放。此外，统一采样隐含地偏向于过时的转换，这些转换是由策略生成的，此后通常会出现数十万次更新。优先重放及其对未见过的转换的奖励直接纠正了第一个问题，并且也往往有助于解决第二个问题，因为最近的转换往往会出现更大的错误——这是因为旧的转换将有更多机会纠正它们，并且因为价值函数往往不太能很好地预测新数据。

我们假设深度神经网络以另一种有趣的方式与优先重放交互。当我们区分学习给定表示的值（即顶层）和学习改进的表示（即底层）时，表示良好的转换将很快减少其错误，然后重放得更少，从而增加对表示较差的其他学习的关注，从而将更多的资源用于区分别名状态 - 如果观察和网络容量允许的话。

6 个扩展

优先监督学习：监督学习背景下优先重放的类似方法是从数据集中进行非均匀采样，每个样本使用基于其上次看到的错误的优先级。这可以帮助将学习集中在那些仍然可以学习的样本上，为（硬）边界情况投入额外的资源，有点类似于提升（Galar et al., 2012）。此外，如果数据集不平衡，我们假设来自稀有类的样本将被不成比例地频繁采样，因为它们的错误缩小得太慢，并且从常见类中选择的样本将是最接近决策边界的样本，从而导致类似于硬负挖掘的效果（Felzenszwalb et al., 2008）。为了检查这些直觉是否成立，我们对经典 MNIST 数字分类问题的类不平衡变体进行了初步实验（LeCun 等人，1998），其中我们删除了训练集中数字 0, 1, 2, 3, 4 的 99% 的样本，同时保持测试/验证集不变（即，保留类平衡）。我们比较两种场景：在知情的情况下，我们人为地重新加权贫困类的误差（100 倍），在不知情的情况下，我们没有提供任何暗示测试分布将与训练分布不同。有关卷积神经网络训练设置的详细信息，请参阅附录 B.3。优先采样（无信息， $\alpha = 1$, $\beta = 0$ ）优于无信息统一基线，并且在泛化方面接近有信息的统一基线（见图 5）；再次，优先训练在学习速度方面也更快。

离策略重放：离策略 RL 的两种标准方法是拒绝采样和使用重要性采样率 ρ 来纠正转换在策略上的可能性。我们的方法包含与这两种方法类似的方法，即重播概率 P 和 IS 校正 w 。因此，如果重放内存中存在可用的转换，那么将其应用于离策略强化学习似乎很自然。特别是，我们使用 $w = \rho$ 、 $\alpha = 0$ 、 $\beta = 1$ 和拒绝采样恢复加权 IS

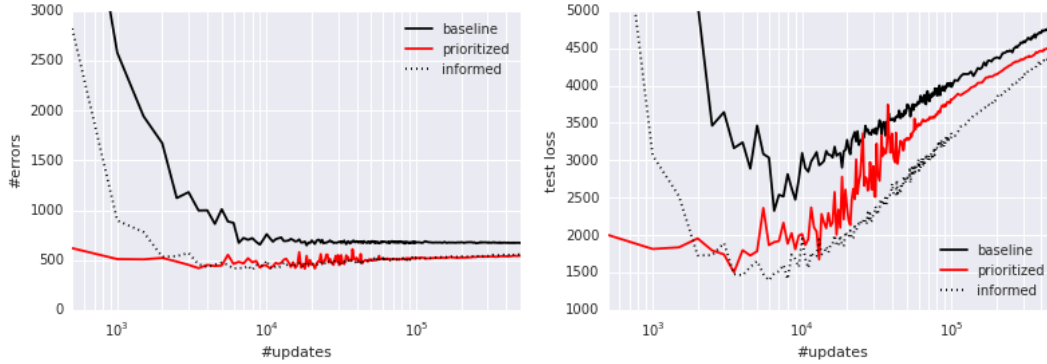


Figure 5: Classification errors as a function of supervised learning updates on severely class-imbalanced MNIST. Prioritized sampling improves performance, leading to comparable errors on the test set, and approaching the imbalance-informed performance (median of 3 random initializations). **Left:** Number of misclassified test set samples. **Right:** Test set loss, highlighting overfitting.

with $p = \min(1; \rho)$, $\alpha = 1$, $\beta = 0$, in the proportional variant. Our experiments indicate that intermediate variants, possibly with annealing or ranking, could be more useful in practice – especially when IS ratios introduce high variance, i.e., when the policy of interest differs substantially from the behavior policy in some states. Of course, off-policy correction is complementary to our prioritization based on expected learning progress, and the same framework can be used for a hybrid prioritization by defining $p = \rho \cdot |\delta|$, or some other sensible trade-off based on both ρ and δ .

Feedback for Exploration: An interesting side-effect of prioritized replay is that the total number M_i that a transition will end up being replayed varies widely, and this gives a rough indication of how useful it was to the agent. This potentially valuable signal can be fed back to the exploration strategy that generates the transitions. For example, we could sample exploration hyperparameters (such as the fraction of random actions ϵ , the Boltzmann temperature, or the amount of intrinsic reward to mix in) from a parametrized distribution at the beginning of each episode, monitor the usefulness of the experience via M_i , and update the distribution toward generating more useful experience. Or, in a parallel system like the Gorila agent (Nair et al., 2015), it could guide resource allocation between a collection of concurrent but heterogeneous ‘actors’, each with different exploration hyperparameters.

Prioritized Memories: Considerations that help determine which transitions to replay are likely to also be relevant for determining which memories to store and when to erase them (e.g. when it becomes likely that they will never be replayed anymore). An explicit control over which memories to keep or erase can help reduce the required total memory size, because it reduces redundancy (frequently visited transitions will have low error, so many of them will be dropped), while automatically adjusting for what has been learned already (dropping many of the ‘easy’ transitions) and biasing the contents of the memory to where the errors remain high. This is a non-trivial aspect, because memory requirements for DQN are currently dominated by the size of the replay memory, no longer by the size of the neural network. Erasing is a more final decision than reducing the replay probability, thus an even stronger emphasis of diversity may be necessary, for example by tracking the age of each transitions and using it to modulate the priority in such a way as to preserve sufficient old experience to prevent cycles (related to ‘hall of fame’ ideas in multi-agent literature, Rosin & Belew, 1997). The priority mechanism is also flexible enough to permit integrating experience from *other sources*, such as from a planner or from human expert trajectories (Guo et al., 2014), since knowing the source can be used to modulate each transition’s priority, e.g. in such a way as to preserve a sufficient fraction of external experience in memory.

7 CONCLUSION

This paper introduced prioritized replay, a method that can make learning from experience replay more efficient. We studied a couple of variants, devised implementations that scale to large replay memories, and found that prioritized replay speeds up learning by a factor 2 and leads to a new state-of-the-art of performance on the Atari benchmark. We laid out further variants and extensions that hold promise, namely for class-imbalanced supervised learning.

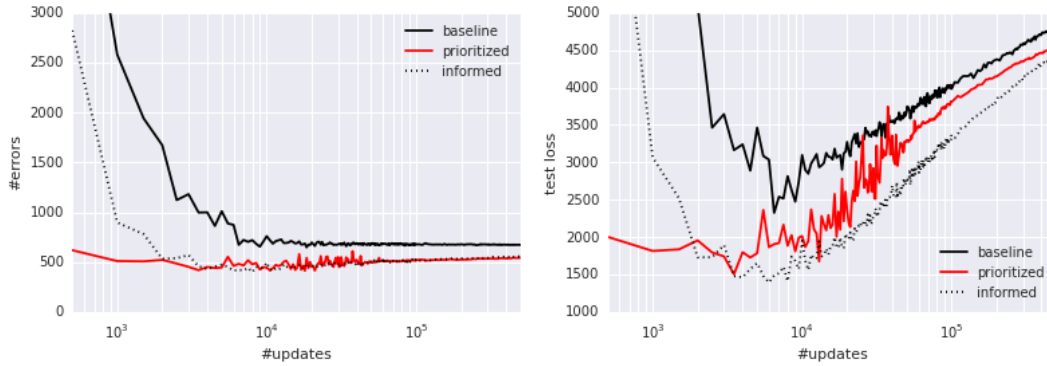


图 5: 分类错误作为严重类别不平衡 MNIST 上监督学习更新的函数。优先采样提高了性能，导致测试集上的可比错误，并接近不平衡通知的性能（3 次随机初始化的中值）。左：错误分类的测试集样本的数量。右：测试集损失，突出显示过度拟合。

在比例变体中， $p = \min(1; \rho)$ 、 $\alpha = 1$ 、 $\beta = 0$ 。我们的实验表明，中间变体（可能带有退火或排序）在实践中可能更有用——特别是当 IS 比率引入高方差时，即当某些州的利益政策与行为政策有很大不同时。当然，离策略修正是对我们基于预期学习进度的优先级的补充，并且可以通过定义 $p = \rho \cdot |\delta|$ 或基于 ρ 和 δ 的其他合理权衡，将相同的框架用于混合优先级。

探索反馈：优先重放的一个有趣的副作用是，最终重放的转换总数 M_i 差异很大，这粗略地表明了它对代理的有用程度。这个潜在有价值的信号可以反馈到生成转换的探索策略。例如，我们可以在每集开始时从参数化分布中采样探索超参数（例如随机动作的分数 ϵ 、玻尔兹曼温度或混合的内在奖励量），通过 M_i 监控体验的有用性，并更新分布以生成更有用的体验。或者，在像 Gorilla 代理（Nair et al., 2015）这样的并行系统中，它可以指导一组并发但异构的“参与者”之间的资源分配，每个参与者都有不同的探索超参数。

优先记忆：有助于确定要重放哪些过渡的考虑因素也可能与确定要存储哪些记忆以及何时删除它们有关（例如，当它们可能永远不会再重放时）。明确控制保留或删除哪些内存可以帮助减少所需的总内存大小，因为它减少了冗余（经常访问的转换将具有较低的错误，因此其中许多将被丢弃），同时自动调整已经学习的内容（丢弃许多“简单”的转换）并将内存内容偏向于错误仍然较高的地方。这是一个重要的方面，因为 DQN 的内存需求目前主要由重放内存的大小决定，不再由神经网络的大小决定。擦除是比降低重放概率更最终的决定，因此可能有必要更加强调多样性，例如通过跟踪每个转换的年龄并使用它来调整优先级，以保留足够的旧经验以防止循环（与多智能体文献中的“名人堂”思想相关，Rosin & Belew, 1997）。优先级机制也足够灵活，允许整合来自 *other sources* 的经验，例如来自规划者或来自人类专家轨迹的经验（Guo 等人, 2014），因为了解源可以用于调整每个转换的优先级，例如以这样的方式在记忆中保留足够部分的外部经验。

7 结论

本文介绍了优先重放，这是一种可以更有效地从经验重放中学习的方法。我们研究了几个变体，设计了可扩展至大型重放内存的实现，并发现优先重放可将学习速度提高 2 倍，并在 Atari 基准测试中带来新的最先进的性能。我们提出了有希望的进一步变体和扩展，即类别不平衡的监督学习。

ACKNOWLEDGMENTS

We thank our Deepmind colleagues, in particular Hado van Hasselt, Joseph Modayil, Nicolas Heess, Marc Bellemare, Razvan Pascanu, Dharshan Kumaran, Daan Wierstra, Arthur Guez, Charles Blundell, Alex Pritzel, Alex Graves, Balaji Lakshminarayanan, Ziyu Wang, Nando de Freitas, Remi Munos and Geoff Hinton for insightful discussions and feedback.

REFERENCES

- Andre, David, Friedman, Nir, and Parr, Ronald. Generalized prioritized sweeping. In *Advances in Neural Information Processing Systems*. Citeseer, 1998.
- Atherton, Laura A, Dupret, David, and Mellor, Jack R. Memory trace replay: the shaping of memory consolidation by neuromodulation. *Trends in neurosciences*, 38(9):560–570, 2015.
- Bellemare, Marc G, Naddaf, Yavar, Veness, Joel, and Bowling, Michael. The arcade learning environment: An evaluation platform for general agents. *arXiv preprint arXiv:1207.4708*, 2012.
- Bellemare, Marc G., Ostrovski, Georg, Guez, Arthur, Thomas, Philip S., and Munos, Rémi. Increasing the action gap: New operators for reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016. URL <http://arxiv.org/abs/1512.04860>.
- Collobert, Ronan, Kavukcuoglu, Koray, and Farabet, Clément. Torch7: A matlab-like environment for machine learning. In *BigLearn, NIPS Workshop*, number EPFL-CONF-192376, 2011.
- Diamond, Jared. Zebras and the Anna Karenina principle. *Natural History*, 103:4–4, 1994.
- Felzenszwalb, Pedro, McAllester, David, and Ramanan, Deva. A discriminatively trained, multi-scale, deformable part model. In *Computer Vision and Pattern Recognition, 2008. CVPR 2008. IEEE Conference on*, pp. 1–8. IEEE, 2008.
- Foster, David J and Wilson, Matthew A. Reverse replay of behavioural sequences in hippocampal place cells during the awake state. *Nature*, 440(7084):680–683, 2006.
- Galar, Mikel, Fernandez, Alberto, Barrenechea, Edurne, Bustince, Humberto, and Herrera, Francisco. A review on ensembles for the class imbalance problem: bagging-, boosting-, and hybrid-based approaches. *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on*, 42(4):463–484, 2012.
- Geramifard, Alborz, Doshi, Finale, Redding, Joshua, Roy, Nicholas, and How, Jonathan. Online discovery of feature dependencies. In *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pp. 881–888, 2011.
- Guo, Xiaoxiao, Singh, Satinder, Lee, Honglak, Lewis, Richard L, and Wang, Xiaoshi. Deep Learning for Real-Time Atari Game Play Using Offline Monte-Carlo Tree Search Planning. In Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N.D., and Weinberger, K.Q. (eds.), *Advances in Neural Information Processing Systems 27*, pp. 3338–3346. Curran Associates, Inc., 2014.
- Hinton, Geoffrey E. To recognize shapes, first learn to generate images. *Progress in brain research*, 165:535–547, 2007.
- Kingma, Diederik P. and Ba, Jimmy. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014.
- Lecun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, Nov 1998. ISSN 0018-9219. doi: 10.1109/5.726791.
- LeCun, Yann, Cortes, Corinna, and Burges, Christopher JC. The MNIST database of handwritten digits, 1998.
- Lin, Long-Ji. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine learning*, 8(3-4):293–321, 1992.
- Mahmood, A Rupam, van Hasselt, Hado P, and Sutton, Richard S. Weighted importance sampling for off-policy learning with linear function approximation. In *Advances in Neural Information Processing Systems*, pp. 3014–3022, 2014.
- McNamara, Colin G, Tejero-Cantero, Álvaro, Trouche, Stéphanie, Campo-Urriza, Natalia, and Dupret, David. Dopaminergic neurons promote hippocampal reactivation and spatial memory persistence. *Nature neuroscience*, 2014.
- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan, and Riedmiller, Martin. Playing atari with deep reinforcement learning. *arXiv preprint*

致谢

我们感谢 Deepmind 的同事，特别是 Hado van Hasselt、Joseph Modayil、Nicolas Heess、Marc Bellemare、Razvan Pascanu、Dharshan Kumaran、Daan Wierstra、Arthur Guez、Charles Blundell、Alex Pritzel、Alex Graves、Balaji Lakshminarayanan、Ziyu Wang、Nando de Freitas、Remi Munos 和 Geoff Hinton 的富有洞察力的讨论和反馈。

参考

安德烈、大卫、弗里德曼、尼尔和帕尔、罗纳德。广义优先清扫。在 *Advances in Neural Information Processing Systems* 中。Citeseer, 1998。Atherton, Laura A, Dupret, David 和 Mellor, Jack R. 记忆痕迹重播：通过神经调节塑造记忆巩固。 *Trends in neurosciences*, 38(9):560–570, 2015。Bellemare, Marc G, Naddaf, Yavar, Veness, Joel 和 Bowling, Michael. 街机学习环境：通用代理的评估平台。 *arXiv preprint arXiv:1207.4708*, 2012。Bellemare, Marc G., Ostrovski, Georg, Guez, Arthur, Thomas, Philip S., 和 Munos, Rémi. 增加行动差距：强化学习的新算子。在 *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016 年。网址 <http://arxiv.org/abs/1512.04860>。罗南·科洛伯特、科雷·卡武克库奥格鲁和克莱门特·法拉贝特。Torch7：类似 matlab 的机器学习环境。在 *BigLearn, NIPS Workshop* 中，编号 EPFL-CONF-192376, 2011。Diamond, Jared. 斑马和安娜卡列尼娜原则。 *Natural History*, 103:4-4, 1994。Felzenszwalb, Pedro, McAllester, David 和 Ramanan, Deva. 经过区分训练的多尺度可变形零件模型。在 *Computer Vision and Pattern Recognition, 2008. CVPR 2008. IEEE Conference on*, 第 1-8 页。IEEE, 2008。大卫·J·福斯特和马修·A·威尔逊。清醒状态下海马位置细胞行为序列的反向重放。 *Nature*, 440(7084):680–683, 2006。Galar, Mikel, Fernandez, Alberto, Barrenechea, Edurne, Bustince, Humberto 和 Herrera, Francisco. 对类不平衡问题的集成的回顾：基于 bagging、boosting 和混合的方法。 *Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on*, 42(4): 463–484, 2012。杰拉米法德、厄尔布尔兹、多西、菲纳莱、雷丁、约书亚、罗伊、尼古拉斯和乔纳森·豪。在线发现特征依赖关系。 *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, 第 881–888 页, 2011 年。Guo, Xiaoxiao, Singh, Satinder, Lee, Honglak, Lewis, Richard L 和 Wang, Xiaoshi. 使用离线蒙特卡罗树搜索规划进行实时 Atari 游戏的深度学习。 Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N.D. 和 Weinberger, K.Q. (编辑), *Advances in Neural Information Processing Systems 27*, 第 3338–3346 页。Curran Associates, Inc., 2014。Hinton, Geoffrey E. 要识别形状，首先要学会生成图像。 *Progress in brain research*, 165:535–547, 2007。Kingma, Diederik P. 和 Ba, Jimmy. Adam：一种随机优化方法。 *CoRR*, abs/1412.6980, 2014。Lecun, Y., Bottou, L., Bengio, Y., and Haffner, P. 基于梯度的学习应用于文档识别。 *Proceedings of the IEEE*, 86(11): 2278–2324, 1998 年 11 月。ISSN 0018-9219. doi: 10.1109/5.726791。LeCun, Yann, Cortes, Corinna 和 Burges, Christopher JC. MNIST 手写数字数据库, 1998。Lin, Long-Ji. 基于强化学习、规划和教学的自我改进反应代理。 *Machine learning*, 8(3-4):293–321, 1992。Mahmood, A Rupam, van Hasselt, Hado P, and Sutton, Richard S. 使用线性函数逼近的离策略学习的加权重要性采样。 *Advances in Neural Information Processing Systems*, 第 3014–3022 页, 2014 年。McNamara, Colin G, Tejero-Cantero, Álvaro, Trouche, Stéphanie, Campo-Urriza, Natalia 和 Dupret, David. 多巴胺能神经元促进海马重新激活和空间记忆持续。 *Nature neuroscience*, 2014 年。Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Graves, Alex, Antonoglou, Ioannis, Wierstra, Daan 和 Riedmiller, Martin. 通过深度强化学习玩 Atari. *arXiv preprint*

- arXiv:1312.5602*, 2013.
- Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A, Veness, Joel, Bellemare, Marc G, Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K, Ostrovski, Georg, Petersen, Stig, Beattie, Charles, Sadik, Amir, Antonoglou, Ioannis, King, Helen, Kumaran, Dharmashan, Wierstra, Daan, Legg, Shane, and Hassabis, Demis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Moore, Andrew W and Atkeson, Christopher G. Prioritized sweeping: Reinforcement learning with less data and less time. *Machine Learning*, 13(1):103–130, 1993.
- Nair, Arun, Srinivasan, Praveen, Blackwell, Sam, Alcicek, Cagdas, Fearon, Rory, Maria, Alessandro De, Panneershelvam, Vedavyas, Suleyman, Mustafa, Beattie, Charles, Petersen, Stig, Legg, Shane, Mnih, Volodymyr, Kavukcuoglu, Koray, and Silver, David. Massively parallel methods for deep reinforcement learning. *arXiv preprint arXiv:1507.04296*, 2015.
- Narasimhan, Karthik, Kulkarni, Tejas, and Barzilay, Regina. Language understanding for text-based games using deep reinforcement learning. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2015.
- Ólafsdóttir, H Freyja, Barry, Caswell, Saleem, Aman B, Hassabis, Demis, and Spiers, Hugo J. Hippocampal place cells construct reward related sequences through unexplored space. *Elife*, 4: e06063, 2015.
- Riedmiller, Martin. Rprop-description and implementation details. 1994.
- Rosin, Christopher D and Belew, Richard K. New methods for competitive coevolution. *Evolutionary Computation*, 5(1):1–29, 1997.
- Schaul, Tom, Zhang, Sixin, and Lecun, Yann. No more pesky learning rates. In *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, pp. 343–351, 2013.
- Schmidhuber, Jürgen. Curious model-building control systems. In *Neural Networks, 1991. 1991 IEEE International Joint Conference on*, pp. 1458–1463. IEEE, 1991.
- Singer, Annabelle C and Frank, Loren M. Rewarded outcomes enhance reactivation of experience in the hippocampus. *Neuron*, 64(6):910–921, 2009.
- Stadie, Bradley C, Levine, Sergey, and Abbeel, Pieter. Incentivizing exploration in reinforcement learning with deep predictive models. *arXiv preprint arXiv:1507.00814*, 2015.
- Sun, Yi, Ring, Mark, Schmidhuber, Jürgen, and Gomez, Faustino J. Incremental basis construction from temporal difference error. In *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, pp. 481–488, 2011.
- van Hasselt, Hado. Double Q-learning. In *Advances in Neural Information Processing Systems*, pp. 2613–2621, 2010.
- van Hasselt, Hado, Guez, Arthur, and Silver, David. Deep Reinforcement Learning with Double Q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016. URL <http://arxiv.org/abs/1509.06461>.
- van Seijen, Harm and Sutton, Richard. Planning by prioritized sweeping with small backups. In *Proceedings of The 30th International Conference on Machine Learning*, pp. 361–369, 2013.
- Wang, Z., de Freitas, N., and Lanctot, M. Dueling network architectures for deep reinforcement learning. Technical report, 2015. URL <http://arxiv.org/abs/1511.06581>.
- Watkins, Christopher JCH and Dayan, Peter. Q-learning. *Machine learning*, 8(3-4):279–292, 1992.
- White, Adam, Modayil, Joseph, and Sutton, Richard S. Surprise and curiosity for big data robotics. In *Workshops at the Twenty-Eighth AAAI Conference on Artificial Intelligence*, 2014.

arXiv:1312.5602, 2013. Mnih, Volodymyr, Kavukcuoglu, Koray, Silver, David, Rusu, Andrei A, Veness, Joel, Bellemare, Marc G, Graves, Alex, Riedmiller, Martin, Fidjeland, Andreas K, Ostrovski, Georg, Petersen, Stig, Beattie, Charles, Sadik, Amir, Antonoglou, Ioannis, King, Helen, 库马兰, 达尔尚, 维尔斯特拉, 达安, 莱格, 谢恩和哈萨比斯, 杰米斯。通过深度强化学习实现人类水平的控制。 *Nature*, 518(7540):529–533, 2015。

Moore, Andrew W 和 Atkeson, Christopher G. 优先清扫: 用更少的数据和更少的时间进行强化学习。 *Machine Learning*, 13(1):103–130, 1993.

Nair, Arun, Srinivasan, Praveen, Blackwell, Sam, Alcicek, Cagdas, Fearon, Rory, Maria, Alessandro De, Panneershelvam, Vedavyas, Suleyman, Mustafa, Beattie, Charles, Petersen, Stig, Legg, Shane, Mnih, 弗拉基米尔, 卡武克库奥卢, 科雷和大卫·西尔弗。深度强化学习的大规模并行方法。 *arXiv preprint arXiv:1507.04296*, 2015 年。

Narasimhan, Karthik, Kulkarni, Tejas 和 Barzilay, Regina。使用深度强化学习对基于文本的游戏进行语言理解。在 *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2015 年。

Olafsdóttir, H Freyja, Barry, Caswell, Saleem, Aman B, Hassabis, Demis 和 Spiers, Hugo J. 海马位置细胞通过未探索的空间构建奖励相关序列。 *Elife*, 4: e06063, 2015。

里德米勒, 马丁。Rprop-描述和实现细节。1994.

Rosin, Christopher D 和 Belew, Richard K. 竞争性协同进化的新方法。 *Evolutionary Computation*, 5(1):1–29, 1997.

Schaul, Tom, Zhang, Sixin 和 Lecun, Yann。不再有令人讨厌的学习率。 *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, 第 343–351 页, 2013 年。

Schmidhuber, Jürgen。好奇的模型构建控制系统。在 *Neural Networks, 1991. 1991 IEEE International Joint Conference on*, 第 1458–1463 页。IEEE, 1991。

Singer, Annabelle C 和 Frank, Loren M. 奖励结果增强了海马体体验的重新激活。 *Neuron*, 64(6):910–921, 2009。

Stadie, Bradley C, Levine, Sergey 和 Abbeel, Pieter。通过深度预测模型激励强化学习的探索。 *arXiv preprint arXiv:1507.00814*, 2015。

Sun, Yi, Ring, Mark, Schmidhuber, Jürgen 和 Gomez, Faustino J. 从时间差误差构建增量基础。见 *Proceedings of the 28th International Conference on Machine Learning (ICML-11)*, 第 481–488 页, 2011 年。

van Hasselt, Hado。双Q学习。 *Advances in Neural Information Processing Systems*, 第 2613–2621 页, 2010 年。

van Hasselt, Hado, Guez, Arthur 和 Silver, David。使用双 Q 学习的深度强化学习。在 *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016 年。网址 <http://arxiv.org/abs/1509.06461>。

哈姆·范·塞金 (van Seijen) 和理查德·萨顿 (Sutton)。通过优先清理和小型备份进行规划。 *Proceedings of The 30th International Conference on Machine Learning*, 第 361–369 页, 2013 年。

Wang, Z., de Freitas, N. 和 Lanctot, M. 用于深度强化学习的决斗网络架构。技术报告, 2015 年。URL <http://arxiv.org/abs/1511.06581>。

克里斯托弗·JCH 沃特金斯和彼得·达扬。Q-学习。 *Machine learning*, 8(3-4):279–292, 1992。

White, Adam, Modayil, Joseph 和 Sutton, Richard S. 对大数据机器人技术的惊喜和好奇心。在 *Workshops at the Twenty-Eighth AAAI Conference on Artificial Intelligence*, 2014 年。

A PRIORITIZATION VARIANTS

The absolute TD-error is only one possible proxy for the ideal priority measure of expected learning progress. While it captures the scale of potential improvement, it ignores inherent stochasticity in rewards or transitions, as well as possible limitations from partial observability or FA capacity; in other words, it is problematic when there are unlearnable transitions. In that case, its *derivative* – which could be approximated by the difference between a transition’s current $|\delta|$ and the $|\delta|$ when it was last replayed³ – may be more useful. This measure is less immediately available, however, and is influenced by whatever was replayed in the meanwhile, which increases its variance. In preliminary experiments, we found it did not outperform $|\delta|$, but this may say more about the class of (near-deterministic) environments we investigated, than about the measure itself.

An orthogonal variant is to consider the norm of the *weight-change* induced by replaying a transition – this can be effective if the underlying optimizer employs adaptive step-sizes that reduce gradients in high-noise directions (Schaul et al., 2013; Kingma & Ba, 2014), thus placing the burden of distinguishing between learnable and unlearnable transitions on the optimizer.

It is possible to modulate prioritization by not treating positive TD-errors the same than negative ones; we can for example invoke the Anna Karenina principle (Diamond, 1994), interpreted to mean that there are many ways in which a transition can be less good than expected, but only one in which can be better, to introduce an *asymmetry* and prioritize positive TD-errors more than negative ones of equal magnitude, because the former are more likely to be informative. Such an asymmetry in replay frequency was also observed in rat studies (Singer & Frank, 2009). Again, our preliminary experiments with such variants were inconclusive.

The evidence from neuroscience suggest that a prioritization based on episodic return rather than expected learning progress may be useful too Atherton et al. (2015); Ólafsdóttir et al. (2015); Foster & Wilson (2006). For this case, we could boost the replay probabilities of entire episodes, instead of transitions, or boost individual transitions by their observed return-to-go (or even their value estimates).

For the issue of preserving sufficient diversity (to prevent overfitting, premature convergence or impoverished representations), there are alternative solutions to our choice of introducing stochasticity, for example, the priorities could be modulated by a novelty measure in observation space. Nothing prevents a hybrid approach where some fraction of the elements (of each minibatch) are sampled according to one priority measure and the rest according to another one, introducing additional diversity. An orthogonal idea is to increase priorities of transitions that have not been replayed for a while, by introducing an explicit *staleness bonus* that guarantees that every transition is revisited from time to time, with that chance increasing at the same rate as its last-seen TD-error becomes stale. In the simple case where this bonus grows linearly with time, this can be implemented at no additional cost by subtracting a quantity proportional to the global step-count from the new priority on any update.⁴

In the particular case of RL with bootstrapping from value functions, it is possible to exploit the sequential structure of the replay memory using the following intuition: a transition that led to a large amount of learning (about its outgoing state) has the potential to change the bootstrapping target for all transitions leading into that state, and thus there is more to be learned about these. Of course we know at least one of these, namely the historic *predecessor* transition, and so boosting its priority makes it more likely to be revisited soon. Similarly to eligibility traces, this lets information trickle backward from a future outcome to the value estimates of the actions and states that led to it. In practice, we add $|\delta|$ of the current transition to predecessor transition’s priority, but only if the predecessor transition is not a terminal one. This idea is related to ‘reverse replay’ observed in rodents Foster & Wilson (2006), and to a recent extension of prioritized sweeping (van Seijen & Sutton, 2013).

³Of course, more robust approximations would be a function of the history of all encountered δ values. In particular, one could imagine an RProp-style update (Riedmiller, 1994) to priorities that increase the priority while the signs match, and reduce it whenever consecutive errors (for the same transition) differ in sign.

⁴If bootstrapping is used with policy iteration, such that the target values come from separate network (as is the case for DQN), then there is a large increase in uncertainty about the priorities when the target network is updated in the outer iteration. At these points, the staleness bonus is increased in proportion to the number of individual (low-level) updates that happened in-between.

A 优先级变体

绝对 TD 误差只是预期学习进度的理想优先级度量的一种可能代理。虽然它捕捉到了潜在改进的规模，但它忽略了奖励或转换中固有的随机性，以及部分可观察性或 FA 能力的可能限制；换句话说，当存在无法学习的转换时，就会出现这个问题。在这种情况下，它的 *derivative*（可以通过过渡的当前 $|\delta|$ 与上次重播时的 $|\delta|^3$ 之间的差异来近似）可能更有用。然而，这种测量方法不太容易立即获得，并且会受到同时重播的内容的影响，从而增加了其方差。在初步实验中，我们发现它的性能并不优于 $|\delta|$ ，但这可能更多地说明了我们研究的（近确定性）环境类别，而不是测量本身。

正交变体是考虑由重放转换引起的 *weight-change* 范数 - 如果底层优化器采用自适应步长来减少高噪声方向的梯度，这可能是有效的（Schaul et al., 2013; Kingma & Ba, 2014），从而将区分可学习和不可学习转换的负担放在优化器上。

通过不将正 TD 误差与负 TD 误差同等对待，可以调整优先级；例如，我们可以援引安娜·卡列尼娜原则（Diamond, 1994），解释为意味着过渡可能有多种方式不如预期，但只有一种方式可以比预期更好，引入 *asymmetry* 并优先考虑正 TD 误差而不是同等大小的负误差，因为前者更有可能提供信息。在大鼠研究中也观察到重播频率的这种不对称性（Singer & Frank, 2009）。同样，我们对此类变体的初步实验并没有得出结论。

来自神经科学的证据表明，基于情景回报而不是预期学习进度的优先级可能也很有用。（2015）；Ólafsdóttir 等人。（2015）；福斯特和威尔逊（2006）。对于这种情况，我们可以提高整个剧集的重播概率，而不是转换，或者通过观察到的返回（甚至是它们的值估计）来提高单个转换。

对于保持足够多样性的问题（以防止过度拟合、过早收敛或贫乏表示），我们选择引入随机性还有其他解决方案，例如，可以通过观察空间中的新颖性度量来调节优先级。没有什么可以阻止混合方法，其中（每个小批量的）元素的某些部分根据一种优先级措施进行采样，其余部分根据另一种优先级措施进行采样，从而引入额外的多样性。正交思想是通过引入显式的 *staleness bonus* 来提高一段时间内未重放的转换的优先级，该显式的 *staleness bonus* 保证每次转换都会不时地重新访问，并且随着最后一次看到的 TD 错误变得过时，机会以相同的速度增加。在这种奖励随时间线性增长的简单情况下，可以通过从任何更新的新优先级中减去与全局步数成比例的数量来实现，无需额外成本。⁴

在从值函数引导的强化学习的特定情况下，可以使用以下直觉来利用重放存储器的顺序结构：导致大量学习（关于其输出状态）的转换有可能改变导致进入该状态的所有转换的引导目标，因此有更多的东西需要学习。当然，我们至少知道其中之一，即历史性的 *predecessor* 过渡，因此提高其优先级使其更有可能很快被重新审视。与资格追踪类似，这可以让信息从未来的结果向后渗透到导致该结果的行动和状态的价值估计。在实践中，我们将当前转换的 $|\delta|$ 添加到前趋转换的优先级，但前提是前趋转换不是终端转换。这个想法与在啮齿类动物 Foster 和 Wilson (2006) 中观察到的“反向重放”以及最近优先清扫的扩展有关（van Seijen 和 Sutton, 2013）。

³Of course, more robust approximations would be a function of the history of all encountered δ values. In particular, one could imagine an RProp-style update (Riedmiller, 1994) to priorities that increase the priority while the signs match, and reduce it whenever consecutive errors (for the same transition) differ in sign.

⁴If bootstrapping is used with policy iteration, such that the target values come from separate network (as is the case for DQN), then there is a large increase in uncertainty about the priorities when the target network is updated in the outer iteration. At these points, the staleness bonus is increased in proportion to the number of individual (low-level) updates that happened in-between.

B EXPERIMENTAL DETAILS

B.1 BLIND CLIFFWALK

For the Blind Cliffwalk experiments (Section 3.1 and following), we use a straight-forward Q-learning (Watkins & Dayan, 1992) setup. The Q-values are represented using either a tabular look-up table, or a linear function approximator, in both cases represented $Q(s, a) := \theta^\top \phi(s, a)$. For each transition, we compute its TD-error using:

$$\delta_t := R_t + \gamma_t \max_a Q(S_t, a) - Q(S_{t-1}, A_{t-1}) \quad (2)$$

and update the parameters using stochastic gradient ascent:

$$\theta \leftarrow \theta + \eta \cdot \delta_t \cdot \nabla_{\theta} Q|_{S_{t-1}, A_{t-1}} = \theta + \eta \cdot \delta_t \cdot \phi(S_{t-1}, A_{t-1}) \quad (3)$$

For the linear FA case we use a very simple encoding of state as a 1-hot vector (as for tabular), but concatenated with a constant bias feature of value 1. To make generalization across actions impossible, we alternate which action is ‘right’ and which one is ‘wrong’ for each state. All elements are initialized with small values near zero, $\theta_i \sim \mathcal{N}(0, 0.1)$.

We vary the size of the problem (number of states n) from 2 to 16. The discount factor is set to $\gamma = 1 - \frac{1}{n}$ which keeps values on approximately the same scale independently of n . This allows us to use a fixed step-size of $\eta = \frac{1}{4}$ in all experiments.

The replay memory is filled by exhaustively executing all 2^n possible sequences of actions until termination (in random order). This guarantees that exactly one sequence will succeed and hit the final reward, and all others will fail with zero reward. The replay memory contains all the relevant experience (the total number of transitions is $2^{n+1} - 2$), at the frequency that it would be encountered when acting online with a random behavior policy. Given this, we can in principle learn until convergence by just increasing the amount of computation; here, convergence is defined as a mean-squared error (MSE) between the Q-value estimates and the ground truth below 10^{-3} .

B.2 ATARI EXPERIMENTS

B.2.1 IMPLEMENTATION DETAILS

Prioritizing using a replay memory with $N = 10^6$ transitions introduced some performance challenges. Here we describe a number of things we did to minimize additional run-time and memory overhead, extending the discussion in Section 3.

Rank-based prioritization Early experiments with Atari showed that maintaining a sorted data structure of 10^6 transitions with constantly changing TD-errors dominated running time. Our final solution was to store transitions in a priority queue implemented with an array-based binary heap. The heap array was then directly used as an approximation of a sorted array, which is infrequently sorted once every 10^6 steps to prevent the heap becoming too unbalanced. This is an unconventional use of a binary heap, however our tests on smaller environments showed learning was unaffected compared to using a perfectly sorted array. This is likely due to the last-seen TD-error only being a proxy for the usefulness of a transition and our use of stochastic prioritized sampling. A small improvement in running time came from avoiding excessive recalculation of partitions for the sampling distribution. We reused the same partition for values of N that are close together and by updating α and β infrequently. Our final implementation for rank-based prioritization produced an additional 2%-4% increase in running time and negligible additional memory usage. This could be reduced further in a number of ways, e.g. with a more efficient heap implementation, but it was good enough for our experiments.

Proportional prioritization The ‘sum-tree’ data structure used here is very similar in spirit to the array representation of a binary heap. However, instead of the usual heap property, the value of a parent node is the sum of its children. Leaf nodes store the transition priorities and the internal nodes are intermediate sums, with the parent node containing the sum over all priorities, p_{total} . This provides a efficient way of calculating the cumulative sum of priorities, allowing $O(\log N)$ updates and sampling. To sample a minibatch of size k , the range $[0, p_{\text{total}}]$ is divided equally into k ranges.

B 实验细节

B.1 盲目悬崖步道

对于盲崖行走实验（第 3.1 节及后续部分），我们使用直接的 Q 学习（Watkins & Dayan, 1992）设置。Q 值使用表格查找表或线性函数逼近器来表示，在两种情况下都表示为 $Q(s, a) := \theta^\top \phi(s, a)$ 。对于每个转换，我们使用以下方法计算其 TD 误差：

$$\delta_t := R_t + \gamma_t \max_a Q(S_t, a) - Q(S_{t-1}, A_{t-1}) \quad (2)$$

并使用随机梯度上升更新参数：

$$\theta \leftarrow \theta + \eta \cdot \delta_t \cdot \nabla_{\theta} Q|_{S_{t-1}, A_{t-1}} = \theta + \eta \cdot \delta_t \cdot \phi(S_{t-1}, A_{t-1}) \quad (3)$$

对于线性 FA 情况，我们使用非常简单的状态编码作为 1-hot 向量（对于表格），但与值为 1 的恒定偏差特征连接。为了使跨动作的泛化变得不可能，我们对每个状态交替选择哪个动作是“正确的”，哪个动作是“错误的”。所有元素均使用接近零的小值 ($\theta_i \sim \mathcal{N}(0, 0.1)$) 进行初始化。

我们将问题的规模（状态数 n ）从 2 更改为 16。折扣因子设置为 $\gamma = 1 - \frac{1}{n}$ ，这使值保持大致相同的规模，独立于 n 。这允许我们在所有实验中使用固定步长 $\eta = \frac{1}{4}$ 。

通过彻底执行所有 2^n 个可能的动作序列直到终止（以随机顺序）来填充重放内存。这保证了只有一个序列会成功并获得最终奖励，而所有其他序列都会失败并获得零奖励。重播内存包含所有相关经验（转换总数为 $2^{n+1} - 2$ ），其频率与随机行为策略在线操作时遇到的频率相同。鉴于此，原则上我们可以通过增加计算量来学习直到收敛；这里，收敛被定义为 Q 值估计与低于 10^{-3} 的真实值之间的均方误差 (MSE)。

B.2 雅达利实验

B.2.1 实施细节

优先使用带有 $N = 10^6$ 转换的重放内存带来了一些性能挑战。在这里，我们描述了为最大限度地减少额外的运行时间和内存开销所做的一些事情，扩展了第 3 节中的讨论。

基于排名的优先级 Atari 的早期实验表明，维护 10^6 转换的排序数据结构以及不断变化的 TD 错误主导了运行时间。我们的最终解决方案是将转换存储在使用基于数组的二进制堆实现的优先级队列中。然后直接使用堆数组作为排序数组的近似值，该数组很少每 10^6 步排序一次，以防止堆变得过于不平衡。这是二进制堆的非常规用法，但是我们在较小环境中的测试表明，与使用完美排序的数组相比，学习不受影响。这可能是由于最后看到的 TD 误差仅代表了过渡的有用性以及我们对随机优先采样的使用。运行时间的小改进来自于避免了对采样分布分区的过度重新计算。我们对接近的 N 值重复使用相同的分区，并且不经常更新 α 和 β 。我们基于排名的优先级的最终实现使运行时间额外增加了 2%-4%，并且额外的内存使用量可以忽略不计。这可以通过多种方式进一步减少，例如具有更高效的堆实现，但对于我们的实验来说已经足够好了。

比例优先级这里使用的“和树”数据结构在精神上与二叉堆的数组表示非常相似。然而，父节点的值不是通常的堆属性，而是其子节点的总和。叶节点存储转换优先级，内部节点是中间总和，父节点包含所有优先级的总和， p_{total} 。这提供了一种计算优先级累积和的有效方法，允许 $O(\log N)$ 更新和采样。为了对大小为 k 的小批量进行采样，范围 $[0, p_{\text{total}}]$ 被平均划分为 k 范围。

Next, a value is uniformly sampled from each range. Finally the transitions that correspond to each of these sampled values are retrieved from the tree. Overhead is similar to rank-based prioritization.

As mentioned in Section 3.4, whenever importance sampling is used, all weights w_i were scaled so that $\max_i w_i = 1$. We found that this worked better in practice as it kept all weights within a reasonable range, avoiding the possibility of extremely large updates. It is worth mentioning that this normalization interacts with annealing on β : as β approaches 1, the normalization constant grows, which reduces the effective average update in a similar way to annealing the step-size η .

B.2.2 HYPERPARAMETERS

Throughout this paper our baseline was DQN and the tuned version of Double DQN. We tuned hyperparameters over a subset of Atari games: Breakout, Pong, Ms. Pac-Man, Q*bert, Alien, Bat-
tlezone, Asterix. Table 2 lists the values tried and Table 3 lists the chosen parameters.

Hyperparameter	Range of values
α	0, 0.4, 0.5, 0.6, 0.7, 0.8
β	0, 0.4, 0.5, 0.6, 1
η	$\eta_{\text{baseline}}, \eta_{\text{baseline}}/2, \eta_{\text{baseline}}/4, \eta_{\text{baseline}}/8$

Table 2: Hyperparameters considered in experiments. Here $\eta_{\text{baseline}} = 0.00025$.

Hyperparameter	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
α (priority)	0	$0.5 \rightarrow 0$	0	0.7	0.6
β (IS)	0	0	0	$0.5 \rightarrow 1$	$0.4 \rightarrow 1$
η (step-size)	0.00025	$\eta_{\text{baseline}}/4$	0.00025	$\eta_{\text{baseline}}/4$	$\eta_{\text{baseline}}/4$

Table 3: Chosen hyperparameters for prioritized variants of DQN. Arrows indicate linear annealing, where the limiting value is reached at the end of training. Note the rank-based variant with DQN as the baseline is an early version without IS. Here, the bias introduced by prioritized replay was instead corrected by annealing α to zero.

B.2.3 EVALUATION

We evaluate our agents using the *human starts* evaluation method described in (van Hasselt et al., 2016). Human starts evaluation uses start states sampled randomly from human traces. The *test* evaluation that agents periodically undergo during training uses start states that are randomized by doing a random number of no-ops at the beginning of each episode. Human starts evaluation averages the score over 100 evaluations of 30 minutes of game time. All learning curve plots show scores under the test evaluation and were generated using the same code base, with the same random seed initializations.

Table 4 and Table 5 show evaluation method differences and the ϵ used in the ϵ -greedy policy for each agent during evaluation. The agent evaluated with the human starts evaluation is the best agent found during training as in (van Hasselt et al., 2016).

Evaluation method	Frames	Emulator time	Number of evaluations	Agent start point
Human starts	108,000	30 mins	100	human starts
Test	500,000	139 mins	1	up to 30 random no-ops

Table 4: Evaluation method comparison.

Normalized score is calculated as in (van Hasselt et al., 2016):

$$\text{score}_{\text{normalized}} = \frac{\text{score}_{\text{agent}} - \text{score}_{\text{random}}}{|\text{score}_{\text{human}} - \text{score}_{\text{random}}|} \quad (4)$$

Note the absolute value of the denominator is taken. This only affects Video Pinball where the random score is higher than the human score. Combined with a high agent score, Video Pinball has

接下来，从每个范围中均匀采样一个值。最后，从树中检索与每个采样值相对应的转换。开销类似于基于排名的优先级。

正如第 3.4 节中提到的，每当使用重要性采样时，所有权重 w_i 都会被缩放，以便 $\max_i w_i = 1$ 。我们发现这在实践中效果更好，因为它将所有权重保持在合理的范围内，避免了极大更新的可能性。值得一提的是，这种归一化与 β 上的退火相互作用：当 β 接近 1 时，归一化常数会增长，这会以与退火步长 η 类似的方式减少有效平均更新。

B.2.2 超参数

在本文中，我们的基线是 DQN 和 Double DQN 的调整版本。我们调整了 Atari 游戏子集的超参数：Breakout、Pong、Ms. Pac-Man、Q*bert、Alien、Battlezone、Asterix。表 2 列出了尝试的值，表 3 列出了所选参数。

Hyperparameter	Range of values
α	0, 0.4, 0.5, 0.6, 0.7, 0.8
β	0, 0.4, 0.5, 0.6, 1
η	$\eta_{\text{baseline}}, \eta_{\text{baseline}}/2, \eta_{\text{baseline}}/4, \eta_{\text{baseline}}/8$

表 2：实验中考虑的超参数。这里是 $\eta_{\text{baseline}} = 0.00025$ 。

Hyperparameter	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
α (priority)	0	$0.5 \rightarrow 0$	0	0.7	0.6
β (IS)	0	0	0	$0.5 \rightarrow 1$	$0.4 \rightarrow 1$
η (step-size)	0.00025	$\eta_{\text{baseline}}/4$	0.00025	$\eta_{\text{baseline}}/4$	$\eta_{\text{baseline}}/4$

表 3：为 DQN 优先变体选择的超参数。箭头表示线性退火，在训练结束时达到极限值。请注意，以 DQN 作为基线的基于排名的变体是没有 IS 的早期版本。在这里，优先重放引入的偏差通过将 α 退火为零来纠正。

B.2.3 评估

我们使用 (van Hasselt et al., 2016) 中描述的 *human starts* 评估方法来评估我们的代理。人类开始评估使用从人类痕迹中随机采样的开始状态。代理在训练期间定期进行的 *test* 评估使用通过在每集开始时执行随机数量的无操作而随机化的开始状态。人类开始评估，对 30 分钟游戏时间的 100 次评估进行平均得分。所有学习曲线图都显示测试评估下的分数，并使用相同的代码库和相同的随机种子初始化生成。

表 4 和表 5 显示了评估方法差异以及评估过程中每个智能体的 ϵ 贪婪策略中使用的 ϵ 。使用人类开始评估进行评估的智能体是在训练期间找到的最佳智能体，如 (van Hasselt et al., 2016) 中所示。

Evaluation method	Frames	Emulator time	Number of evaluations	Agent start point
Human starts	108,000	30 mins	100	human starts
Test	500,000	139 mins	1	up to 30 random no-ops

表 4：评价方法比较。

标准化分数的计算方式如下 (van Hasselt et al., 2016)：

$$\text{score}_{\text{normalized}} = \frac{\text{score}_{\text{agent}} - \text{score}_{\text{random}}}{|\text{score}_{\text{human}} - \text{score}_{\text{random}}|} \quad (4)$$

请注意，取分母的绝对值。这仅影响随机得分高于人类得分的视频弹球游戏。结合高代理分数，视频弹球有

Evaluation method	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Human starts	0.05	0.01	0.001	0.001	0.001
Test	0.05	0.05	0.01	0.01	0.01

Table 5: The ϵ used in the ϵ -greedy policy for each agent, for each evaluation method.

a large effect on the mean normalized score. We continue to use this so our normalized scores are comparable.

B.3 CLASS-IMBALANCED MNIST

B.3.1 DATASET SETUP

In our supervised learning setting we modified MNIST to obtain a new training dataset with a significant label imbalance. This new dataset was obtained by considering a small subset of the samples that correspond to the first 5 digits (0, 1, 2, 3, 4) and all of the samples that correspond to the remaining 5 labels (5, 6, 7, 8, 9). For each of the first 5 digits we randomly sampled 1% of the available examples, i.e., 1% of the available 0s, 1% of the available 1s etc. In the resulting dataset there are examples of all 10 different classes but it is highly imbalanced since there are 100 times more examples that correspond to the 5, 6, 7, 8, 9 classes than to the 0, 1, 2, 3, 4 ones. In all our experiments we used the original MNIST *test* dataset without removing any samples.

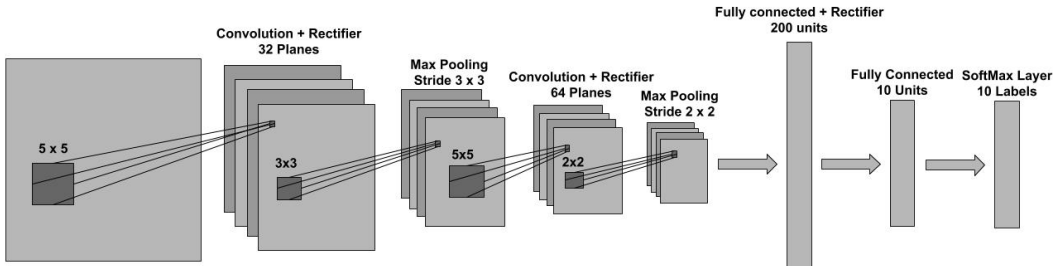


Figure 6:

The architecture of the feed-forward network used in the Prioritized Supervised Learning experiments.

B.3.2 TRAINING SETUP

In our experiments we used a 4 layer feed-forward neural network with an architecture similar to that of LeNet5 (Lecun et al., 1998). This is a 2 layer convolutional neural network followed by 2 fully connected layers at the top. Each convolutional layer is comprised of a pure convolution followed by a rectifier non-linearity and a subsampling max pooling operation. The two fully-connected layers in the network are also separated by a rectifier non-linearity. The last layer is a softmax which is used to obtain a normalized distribution over the possible labels. The complete architecture is shown in Figure 6, and is implemented using Torch7 (Collobert et al., 2011). The model was trained using stochastic gradient descent with no momentum and a minibatch of size 60. In all our experiments we considered 6 different step-sizes (0.3, 0.1, 0.03, 0.01, 0.003 and 0.001) and for each case presented in this work, we selected the step-size that lead to the best (balanced) validation performance. We used the negative log-likelihood loss criterion and we ran experiments with both the weighted and unweighted version of the loss. In the weighted case the loss of the examples that correspond to the first 5 digits (0, 1, 2, 3, 4) was scaled by a factor of a 100 to accommodate the label imbalance in the training set described above.

Evaluation method	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Human starts	0.05	0.01	0.001	0.001	0.001
Test	0.05	0.05	0.01	0.01	0.01

表 5: 对于每种评估方法, 每个代理的 ϵ 贪婪策略中使用的 ϵ_0 。

对平均归一化分数有很大影响。我们继续使用它, 以便我们的标准化分数具有可比性。

B.3 类不平衡 MNIST

B.3.1 数据集设置

在我们的监督学习环境中, 我们修改了 MNIST 以获得具有显著标签不平衡的新训练数据集。这个新数据集是通过考虑对应于前 5 个数字 (0、1、2、3、4) 的一小部分样本以及对应于其余 5 个标签 (5、6、7、8、9) 的所有样本而获得的。对于前 5 位数字中的每一个, 我们随机抽取 1% 的可用示例, 即 1% 的可用 0、1% 的可用 1 等。在结果数据集中, 有所有 10 个不同类别的示例, 但它是高度不平衡的, 因为对应于 5、6、7、8、9 类的示例比对应于 0、1、2、3 的示例多 100 倍。4 个。在我们所有的实验中, 我们使用原始的 MNIST *test* 数据集, 而不删除任何样本。

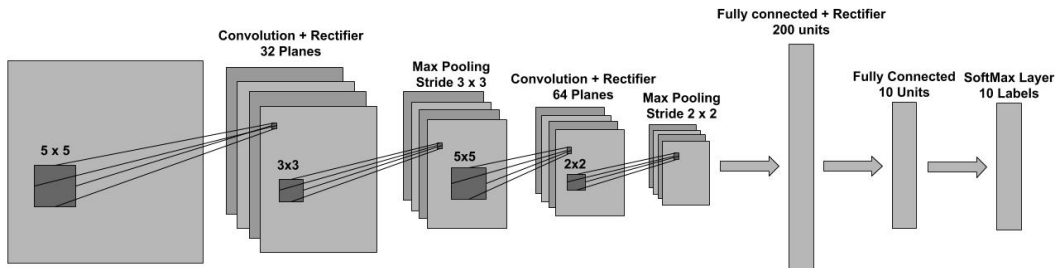


图6:

优先监督学习实验中使用的前馈网络架构。

B.3.2 训练设置

在我们的实验中, 我们使用了 4 层前馈神经网络, 其架构类似于 LeNet5 (Lecun 等人, 1998)。这是一个 2 层卷积神经网络, 顶部有 2 个全连接层。每个卷积层都由纯卷积、非线性整流器和子采样最大池化操作组成。网络中的两个全连接层也被非线性整流器分开。最后一层是 softmax, 用于获取可能标签的归一化分布。完整的架构如图 6 所示, 并使用 Torch7 实现 (Collobert 等人, 2011)。该模型使用无动量和大小为 60 的小批量的随机梯度下降进行训练。在我们所有的实验中, 我们考虑了 6 种不同的步长 (0.3、0.1、0.03、0.01、0.003 和 0.001), 对于本工作中提出的每种情况, 我们选择了能够实现最佳 (平衡) 验证性能的步长。我们使用负对数似然损失准则, 并对损失的加权和未加权版本进行了实验。在加权情况下, 对应于前 5 位数字 (0、1、2、3、4) 的示例损失按 100 倍缩放, 以适应上述训练集中的标签不平衡。

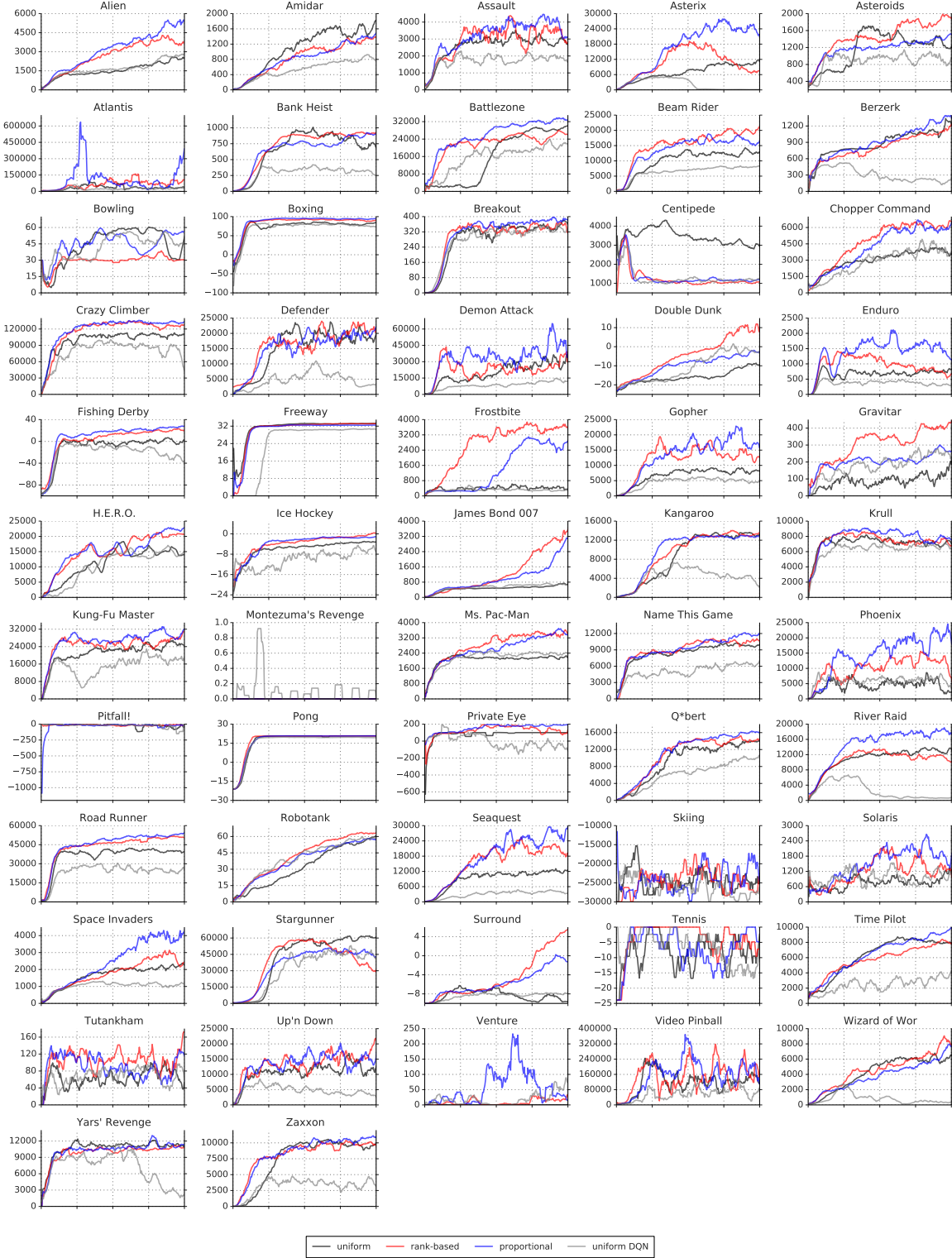


Figure 7: Learning curves (in raw score) for Double DQN (uniform baseline, in black), with rank-based prioritized replay (red), proportional prioritization (blue), for all 57 games of the Atari benchmark suite. Each curve corresponds to a single training run over 200 million unique frames, using test evaluation (see Section B.2.3), with a moving average smoothed over 10 points. Learning curves for the original DQN are in gray. See Figure 8 for a more detailed view on a subset of these.

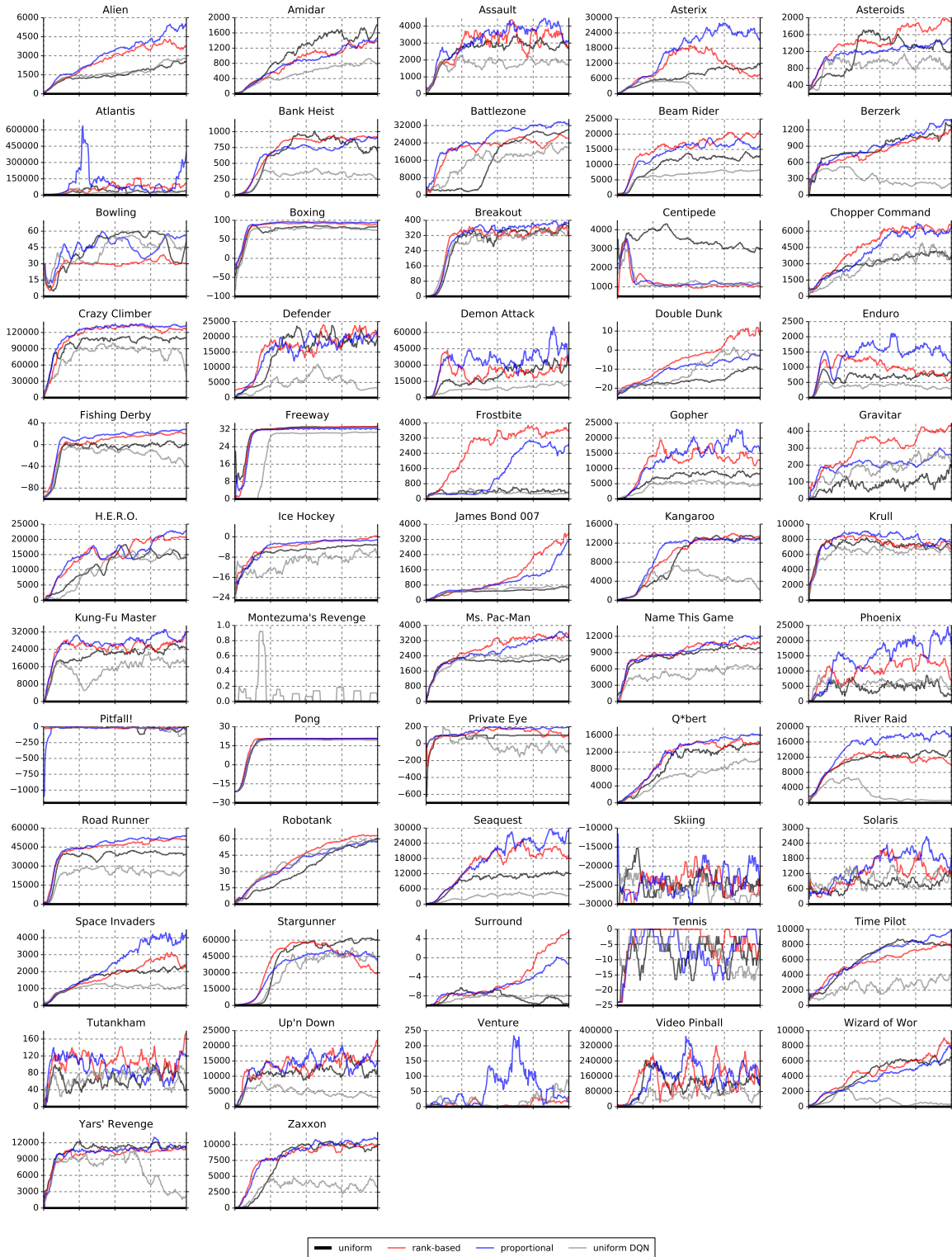


图 7: 对于 Atari 基准套件的所有 57 款游戏, 双 DQN (统一基线, 黑色) 的学习曲线 (以原始分数表示), 具有基于排名的优先重播 (红色)、比例优先级 (蓝色)。每条曲线对应于超过 2 亿个独特帧的单次训练, 使用测试评估 (参见第 B.2.3 节), 移动平均值平滑超过 10 个点。原始 DQN 的学习曲线为灰色。有关其中一部分的更详细视图, 请参见图 8。

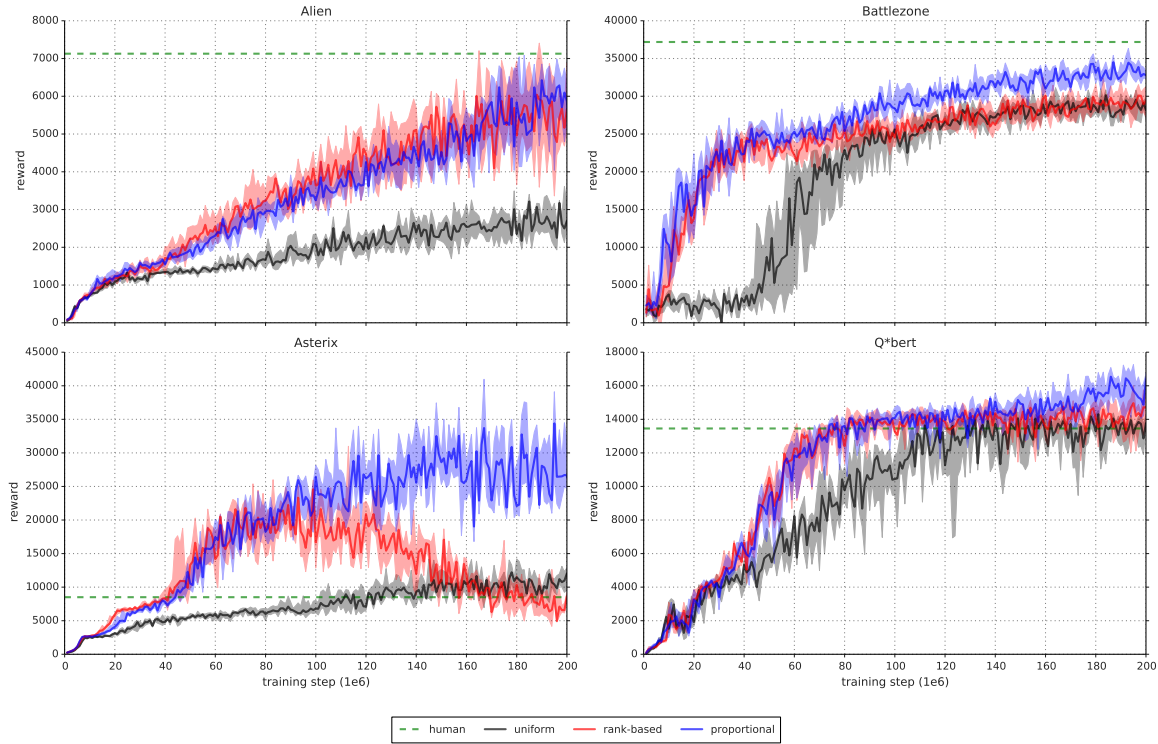


Figure 8: Detailed learning curves for rank-based (red) and proportional (blue) prioritization, as compared to the uniform Double DQN baseline (black) on a selection of games. The solid lines are the median scores, and the shaded area denotes the interquartile range across 8 random initializations. The dashed green lines are human scores. While the variability between runs is substantial, there are significant differences in final achieved score, and also in learning speed.

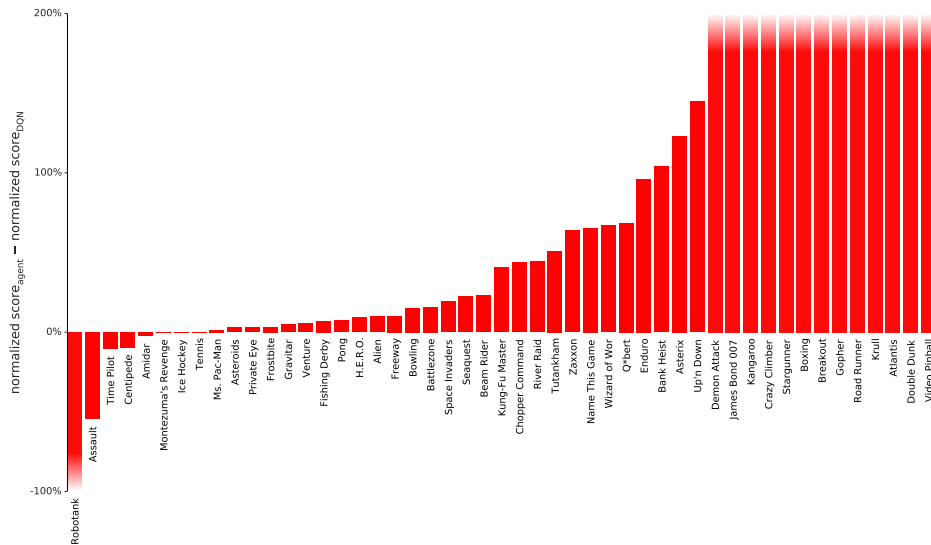


Figure 9: Difference in normalized score (the gap between random and human is 100%) on 49 games with human starts, comparing DQN with and without rank-based prioritized replay, showing substantial improvements in many games. Exact scores are in Table 6. See also Figure 3 where Double DQN is the baseline.

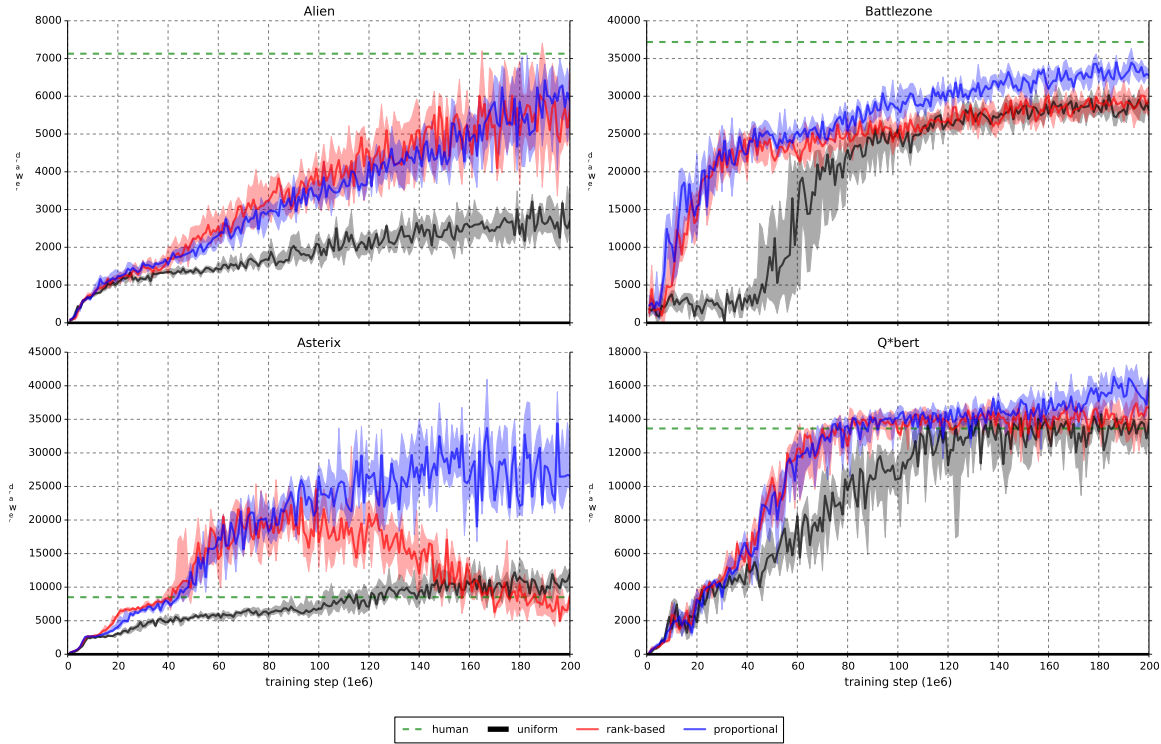


图 8: 基于排名 (红色) 和比例 (蓝色) 优先级的详细学习曲线, 与精选游戏的统一双 DQ N 基线 (黑色) 相比。实线是中位数分数, 阴影区域表示 8 个随机初始化的四分位数范围。绿色虚线是人类得分。虽然跑步之间的差异很大, 但最终获得的分数以及学习速度也存在显著差异。

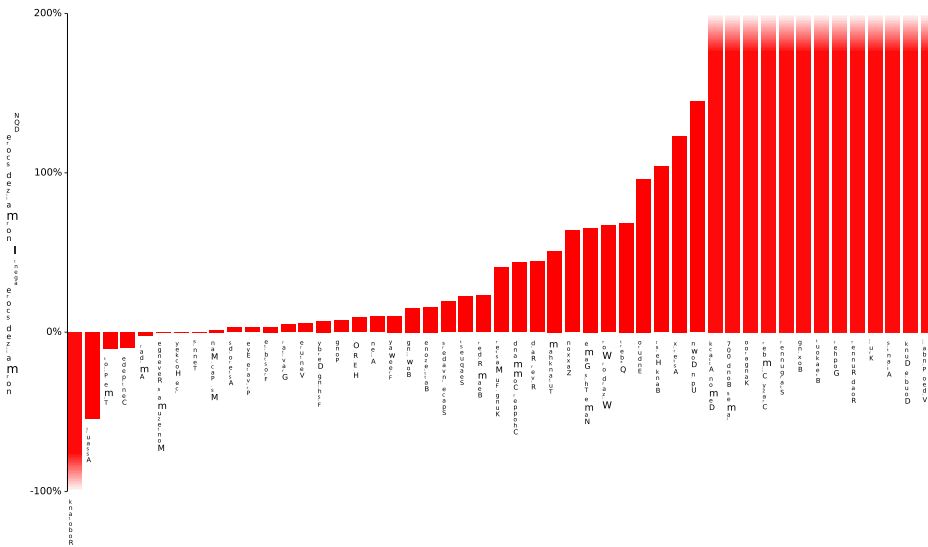


图 9: 在 49 场人类开始的游戏, 标准化分数的差异 (随机分数与人类分数之间的差距为 100%), 比较了有和没有基于排名的优先重放的 DQN, 显示许多游戏有显著的改进。确切的分数如表 6 所示。另请参见图 3, 其中 Double DQN 是基线。

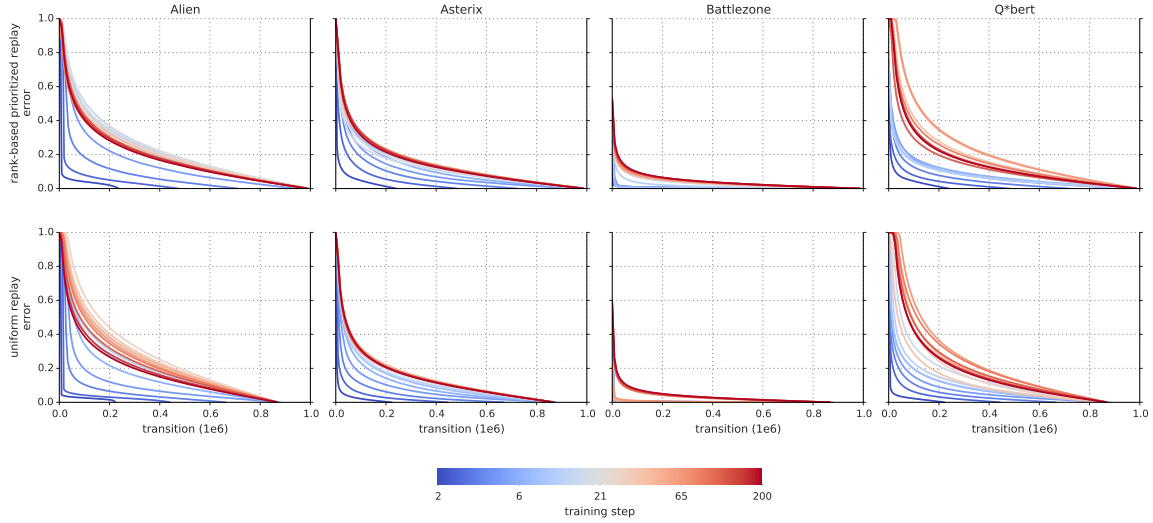


Figure 10: Visualization of the last-seen absolute TD-error for all transitions in the replay memory, sorted, for a selection of Atari games. The lines are color-coded by the time during learning, at a resolution of 10^6 frames, with the coldest colors in the beginning and the warmest toward the end of training. We observe that in some games it starts quite peaked but quickly becomes spread out, following approximately a heavy-tailed distribution. This phenomenon happens for both rank-based prioritized replay (top) and uniform replay (bottom) but is faster for prioritized replay.

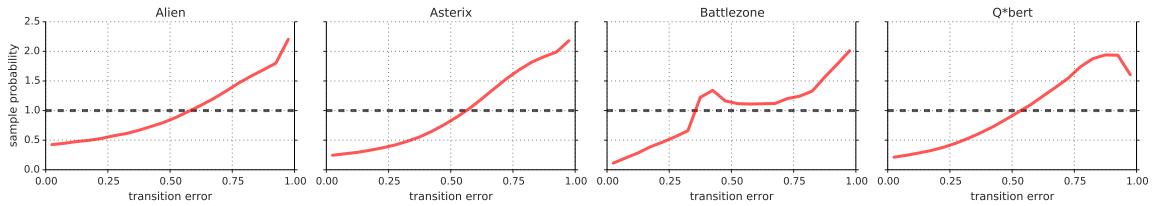


Figure 11: **Effective replay probability**, as a function of absolute TD-error, for the rank-based prioritized replay variant near the start of training. This shows the effect of Equation 1 with $\alpha = 0.7$ in practice, compared to the uniform baseline (dashed horizontal line). The effect is irregular, but qualitatively similar for a selection of games.

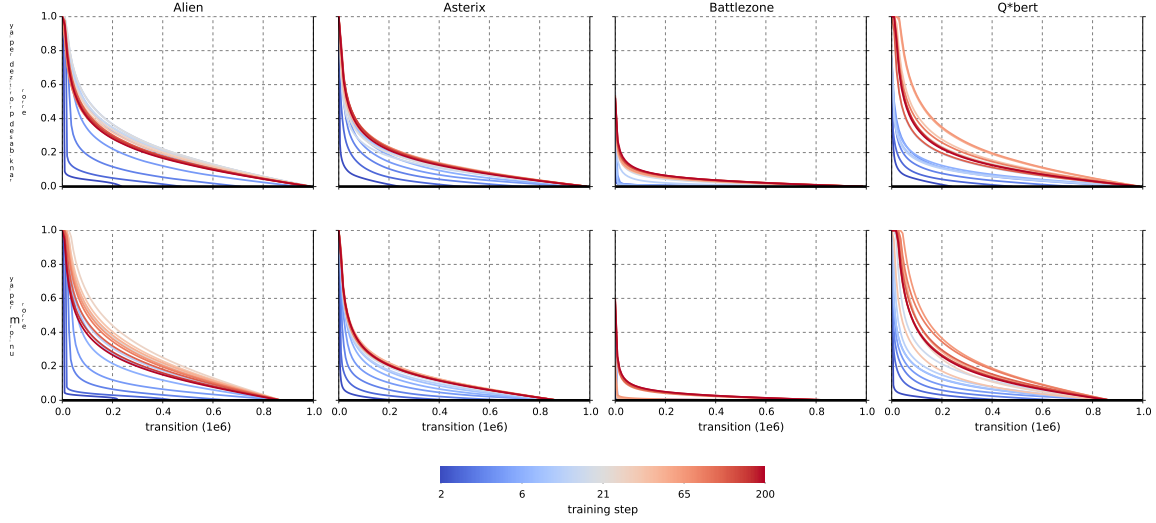


图 10: 精选 Atari 游戏的重播内存中所有转换的最后看到的绝对 TD 误差的可视化（已排序）。这些线条在学习期间按时间进行颜色编码，分辨率为 10^6 帧，在训练开始时使用最冷的颜色，在训练结束时使用最暖的颜色。我们观察到，在某些游戏中，它一开始就达到峰值，但很快就变得分散，大致遵循重尾分布。这种现象在基于排名的优先重放（顶部）和统一重放（底部）中都会发生，但优先重放速度更快。

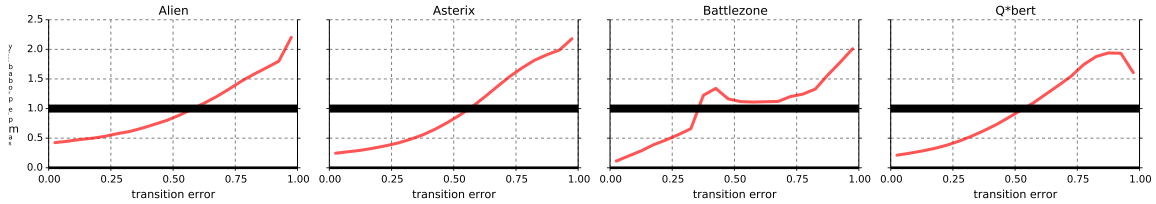


图 11: 训练开始时基于排名的优先重放变体的有效重放概率，作为绝对 TD 误差的函数。这显示了与统一基线（水平虚线）相比，方程 1 在实践中使用 $\alpha = 0.7$ 的效果。效果是不规则的，但对于某些游戏来说在性质上是相似的。

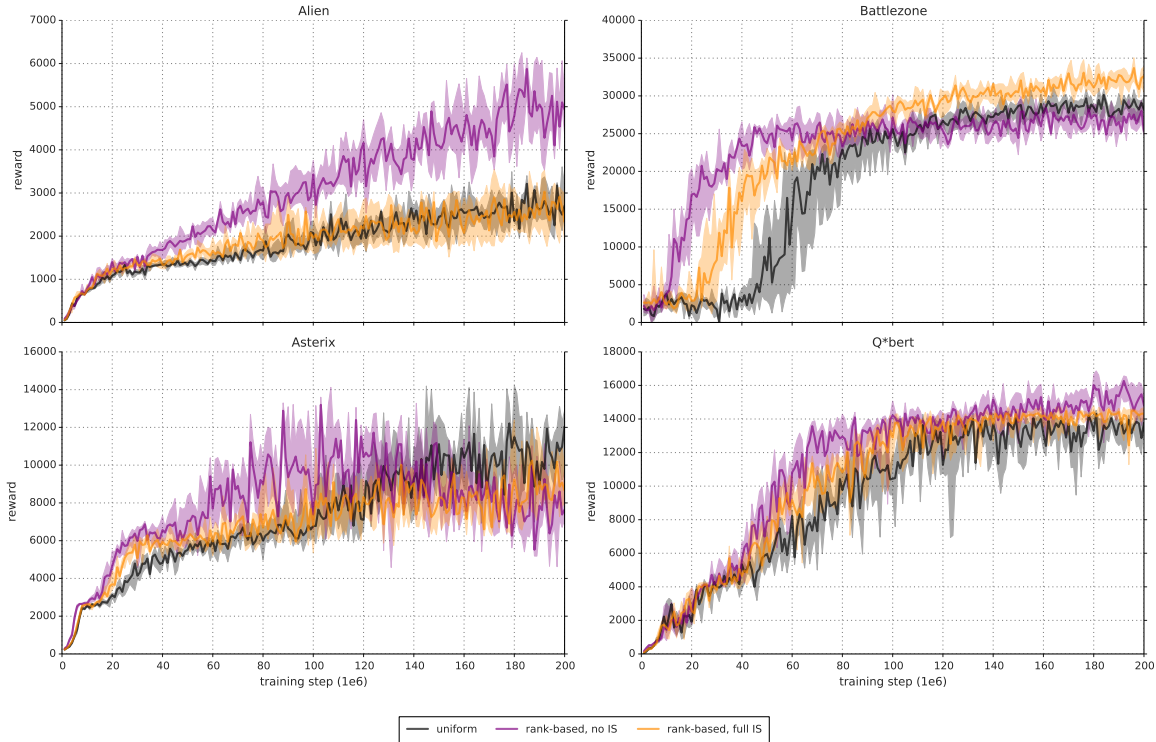


Figure 12: **Effect of importance sampling:** These learning curves (as in Figure 8) show how rank-based prioritization is affected by full importance-sampling correction (i.e., $\beta = 1$, in orange), as compared to the uniform baseline (black, $\alpha = 0$) and pure, uncorrected prioritized replay (violet, $\beta = 0$), on a few selected games. The shaded area corresponds to the interquartile range. The step-size for full IS correction is the same as for uniform replay. For uncorrected prioritized replay, the step-size is reduced by a factor of 4. Compared to uncorrected prioritized replay, importance sampling makes learning less aggressive, leading on the one hand to slower initial learning, but on the other hand to a smaller risk of premature convergence and sometimes better ultimate results. Compared to uniform replay, fully corrected prioritization is on average better.

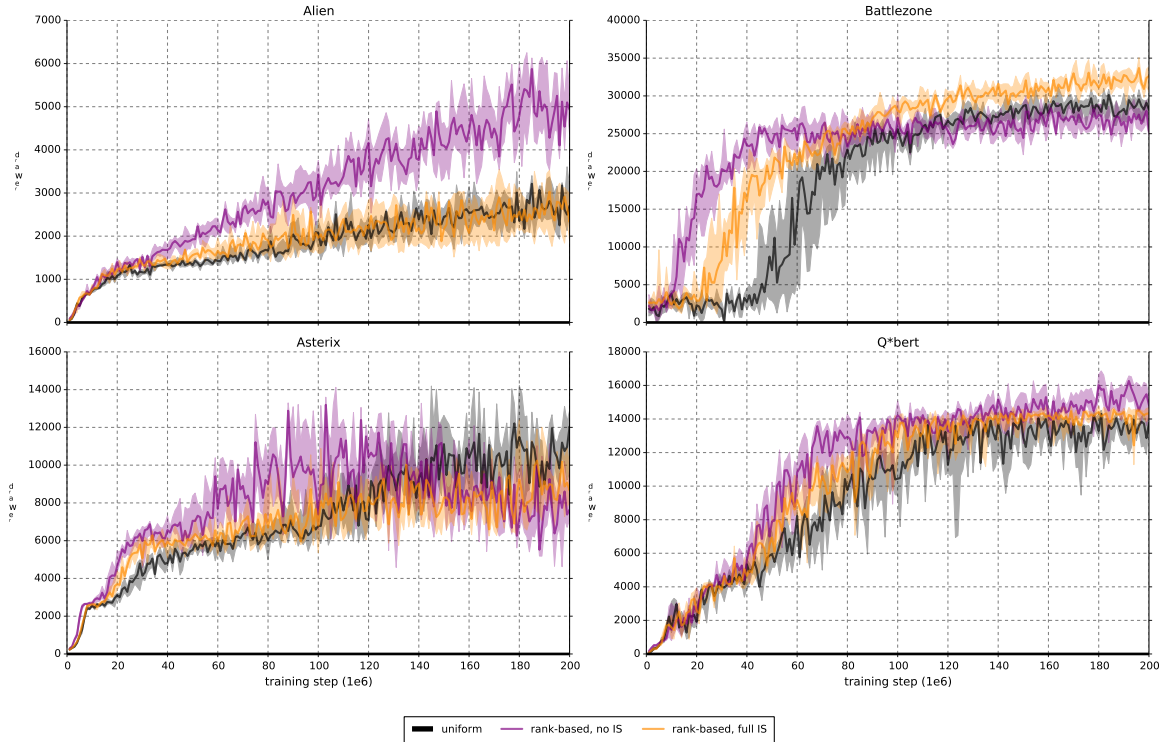


图 12: 重要性采样的效果: 这些学习曲线 (如图 8 所示) 显示了在一些选定的游戏中, 与统一基线 (黑色, $\alpha = 0$) 和纯粹的、未校正的优先重放 (紫色, $\beta = 0$) 相比, 完全重要性采样校正 (即 $\beta = 1$, 橙色) 如何影响基于排名的优先级。阴影区域对应于四分位数范围。完整 IS 校正的步长与均匀重放的步长相同。对于未校正的优先级重放, 步长减小了 4 倍。与未校正的优先级重放相比, 重要性采样使学习变得不那么激进, 一方面导致初始学习速度变慢, 但另一方面导致过早收敛的风险更小, 有时最终结果更好。与统一重播相比, 完全校正的优先级平均而言更好。

Game	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Alien	7%	17%	14%	19%	12%
Amidar	8%	6%	10%	8%	14%
Assault	685%	631%	1276%	1381%	1641%
Asterix	-1%	123%	226%	303%	431%
Asteroids	-0%	2%	1%	2%	2%
Atlantis	478%	1480%	2335%	2419%	4425%
Bank Heist	25%	129%	139%	137%	128%
Battlezone	48%	63%	72%	75%	87%
Beam Rider	57%	80%	117%	210%	176%
Berzerk		22%	40%	33%	47%
Bowling	5%	20%	31%	15%	27%
Boxing	246%	641%	676%	665%	632%
Breakout	1149%	1823%	1397%	1298%	1407%
Centipede	22%	12%	23%	19%	18%
Chopper Command	29%	73%	34%	48%	72%
Crazy Climber	179%	429%	448%	507%	522%
Defender		151%	207%	176%	155%
Demon Attack	390%	596%	2152%	1888%	2256%
Double Dunk	-350%	669%	981%	2000%	1169%
Enduro	68%	164%	158%	233%	239%
Fishing Derby	91%	98%	98%	106%	105%
Freeway	101%	111%	113%	113%	109%
Frostbite	2%	5%	33%	83%	69%
Gopher	120%	836%	728%	1679%	2792%
Gravitar	-1%	4%	-2%	1%	-1%
H.E.R.O.	47%	56%	55%	80%	78%
Ice Hockey	58%	58%	71%	93%	85%
James Bond 007	94%	311%	161%	1172%	1038%
Kangaroo	98%	339%	421%	458%	384%
Krull	283%	1051%	590%	598%	653%
Kung-Fu Master	56%	97%	146%	153%	151%
Montezuma's Revenge	1%	0%	0%	1%	-0%
Ms. Pac-Man	4%	5%	7%	11%	11%
Name This Game	73%	138%	143%	173%	200%
Phoenix		270%	202%	284%	474%
Pitfall!		2%	3%	-1%	5%
Pong	102%	110%	111%	110%	110%
Private Eye	-1%	2%	-2%	0%	-1%
Q*bert	37%	106%	91%	82%	93%
River Raid	25%	70%	74%	81%	128%
Road Runner	136%	854%	643%	780%	850%
Robotank	863%	752%	872%	828%	815%
Seaquest	6%	29%	36%	63%	97%
Skiing		-122%	33%	44%	38%
Solaris		-21%	-14%	3%	2%
Space Invaders	99%	118%	191%	291%	693%
Stargunner	378%	660%	653%	689%	580%
Surround		29%	77%	103%	58%
Tennis	130%	130%	93%	110%	132%
Time Pilot	100%	89%	140%	113%	176%
Tutankham	16%	67%	63%	35%	17%
Up'n Down	28%	173%	200%	125%	313%
Venture	4%	9%	0%	7%	22%
Video Pinball	-5%	4042%	7221%	5727%	7367%
Wizard of Wor	-15%	52%	144%	131%	177%
Yars' Revenge		11%	10%	7%	10%
Zaxxon	4%	68%	102%	113%	113%

Table 6: Normalized scores on 57 Atari games (random is 0%, human is 100%), from a single training run each, using human starts evaluation (see Section B.2.3). Baselines are from van Hasselt et al. (2016), see Equation 4 for how normalized scores are calculated.

Game	DQN		Double DQN (tuned)		
	baseline	rank-based	baseline	rank-based	proportional
Alien	7%	17%	14%	19%	12%
Amidar	8%	6%	10%	8%	14%
Assault	685%	631%	1276%	1381%	1641%
Asterix	-1%	123%	226%	303%	431%
Asteroids	-0%	2%	1%	2%	2%
Atlantis	478%	1480%	2335%	2419%	4425%
Bank Heist	25%	129%	139%	137%	128%
Battlezone	48%	63%	72%	75%	87%
Beam Rider	57%	80%	117%	210%	176%
Berzerk		22%	40%	33%	47%
Bowling	5%	20%	31%	15%	27%
Boxing	246%	641%	676%	665%	632%
Breakout	1149%	1823%	1397%	1298%	1407%
Centipede	22%	12%	23%	19%	18%
Chopper Command	29%	73%	34%	48%	72%
Crazy Climber	179%	429%	448%	507%	522%
Defender		151%	207%	176%	155%
Demon Attack	390%	596%	2152%	1888%	2256%
Double Dunk	-350%	669%	981%	2000%	1169%
Enduro	68%	164%	158%	233%	239%
Fishing Derby	91%	98%	98%	106%	105%
Freeway	101%	111%	113%	113%	109%
Frostbite	2%	5%	33%	83%	69%
Gopher	120%	836%	728%	1679%	2792%
Gravitar	-1%	4%	-2%	1%	-1%
H.E.R.O.	47%	56%	55%	80%	78%
Ice Hockey	58%	58%	71%	93%	85%
James Bond 007	94%	311%	161%	1172%	1038%
Kangaroo	98%	339%	421%	458%	384%
Krull	283%	1051%	590%	598%	653%
Kung-Fu Master	56%	97%	146%	153%	151%
Montezuma's Revenge	1%	0%	0%	1%	-0%
Ms. Pac-Man	4%	5%	7%	11%	11%
Name This Game	73%	138%	143%	173%	200%
Phoenix		270%	202%	284%	474%
Pitfall!		2%	3%	-1%	5%
Pong	102%	110%	111%	110%	110%
Private Eye	-1%	2%	-2%	0%	-1%
Q*bert	37%	106%	91%	82%	93%
River Raid	25%	70%	74%	81%	128%
Road Runner	136%	854%	643%	780%	850%
Robotank	863%	752%	872%	828%	815%
Seaquest	6%	29%	36%	63%	97%
Skiing		-122%	33%	44%	38%
Solaris		-21%	-14%	3%	2%
Space Invaders	99%	118%	191%	291%	693%
Stargunner	378%	660%	653%	689%	580%
Surround		29%	77%	103%	58%
Tennis	130%	130%	93%	110%	132%
Time Pilot	100%	89%	140%	113%	176%
Tutankham	16%	67%	63%	35%	17%
Up'n Down	28%	173%	200%	125%	313%
Venture	4%	9%	0%	7%	22%
Video Pinball	-5%	4042%	7221%	5727%	7367%
Wizard of Wor	-15%	52%	144%	131%	177%
Yars' Revenge		11%	10%	7%	10%
Zaxxon	4%	68%	102%	113%	113%

表 6: 57 个 Atari 游戏的归一化分数（随机为 0%，人类为 100%），每个游戏来自一次训练，使用人类开始评估（参见 B.2.3 节）。基线来自 van Hasselt 等人。 (2016)，请参阅公式 4 了解标准化分数的计算方式。

Game			DQN			Double DQN (tuned)		
	random	human	baseline	Gorila	rank-b.	baseline	rank-b.	prop.
Alien	128.3	6371.3	570.2	813.5	1191.0	1033.4	1334.7	900.5
Amidar	11.8	1540.4	133.4	189.2	98.9	169.1	129.1	218.4
Assault	166.9	628.9	3332.3	1195.8	3081.3	6060.8	6548.9	7748.5
Asterix	164.5	7536.0	124.5	3324.7	9199.5	16837.0	22484.5	31907.5
Asteroids	871.3	36517.3	697.1	933.6	1677.2	1193.2	1745.1	1654.0
Atlantis	13463.0	26575.0	76108.0	629166.5	207526.0	319688.0	330647.0	593642.0
Bank Heist	21.7	644.5	176.3	399.4	823.7	886.0	876.6	816.8
Battlezone	3560.0	33030.0	17560.0	19938.0	22250.0	24740.0	25520.0	29100.0
Beam Rider	254.6	14961.0	8672.4	3822.1	12041.9	17417.2	31181.3	26172.7
Berzerk	196.1	2237.5			644.0	1011.1	865.9	1165.6
Bowling	35.2	146.5	41.2	54.0	58.0	69.6	52.0	65.8
Boxing	-1.5	9.6	25.8	74.2	69.6	73.5	72.3	68.6
Breakout	1.6	27.9	303.9	313.0	481.1	368.9	343.0	371.6
Centipede	1925.5	10321.9	3773.1	6296.9	2959.4	3853.5	3489.1	3421.9
Chopper Command	644.0	8930.0	3046.0	3191.8	6685.0	3495.0	4635.0	6604.0
Crazy Climber	9337.0	32667.0	50992.0	65451.0	109337.0	113782.0	127512.0	131086.0
Defender	1965.5	14296.0			20634.0	27510.0	23666.5	21093.5
Demon Attack	208.3	3442.8	12835.2	14880.1	19478.8	69803.4	61277.5	73185.8
Double Dunk	-16.0	-14.4	-21.6	-11.3	-5.3	-0.3	16.0	2.7
Enduro	-81.8	740.2	475.6	71.0	1265.6	1216.6	1831.0	1884.4
Fishing Derby	-77.1	5.1	-2.3	4.6	3.5	3.2	9.8	9.2
Freeway	0.1	25.6	25.8	10.2	28.4	28.8	28.9	27.9
Frostbite	66.4	4202.8	157.4	426.6	288.7	1448.1	3510.0	2930.2
Gopher	250.0	2311.0	2731.8	4373.0	17478.2	15253.0	34858.8	57783.8
Gravitar	245.5	3116.0	216.5	538.4	351.0	200.5	269.5	218.0
H.E.R.O.	1580.3	25839.4	12952.5	8963.4	15150.9	14892.5	20889.9	20506.4
Ice Hockey	-9.7	0.5	-3.8	-1.7	-3.8	-2.5	-0.2	-1.0
James Bond 007	33.5	368.5	348.5	444.0	1074.5	573.0	3961.0	3511.5
Kangaroo	100.0	2739.0	2696.0	1431.0	9053.0	11204.0	12185.0	10241.0
Krull	1151.9	2109.1	3864.0	6363.1	11209.5	6796.1	6872.8	7406.5
Kung-Fu Master	304.0	20786.8	11875.0	20620.0	20181.0	30207.0	31676.0	31244.0
Montezuma's Revenge	25.0	4182.0	50.0	84.0	44.0	42.0	51.0	13.0
Ms. Pac-Man	197.8	15375.0	763.5	1263.0	964.7	1241.3	1865.9	1824.6
Name This Game	1747.8	6796.0	5439.9	9238.5	8738.5	8960.3	10497.6	11836.1
Phoenix	1134.4	6686.2			16107.8	12366.5	16903.6	27430.1
Pitfall!	-348.8	5998.9			-193.7	-186.7	-427.0	-14.8
Pong	-18.0	15.5	16.2	16.7	18.7	19.1	18.9	18.9
Private Eye	662.8	64169.1	298.2	2598.6	2202.3	-575.5	670.7	179.0
Q*bert	183.0	12085.0	4589.8	7089.8	12740.5	11020.8	9944.0	11277.0
River Raid	588.3	14382.2	4065.3	5310.3	10205.5	10838.4	11807.2	18184.4
Road Runner	200.0	6878.0	9264.0	43079.8	57207.0	43156.0	52264.0	56990.0
Robotank	2.4	8.9	58.5	61.8	51.3	59.1	56.2	55.4
Seaquest	215.5	40425.8	2793.9	10145.9	11848.8	14498.0	25463.7	39096.7
Skiing	-15287.4	-3686.6			-29404.3	-11490.4	-10169.1	-10852.8
Solaris	2047.2	11032.6			134.6	810.0	2272.8	2238.2
Space Invaders	182.6	1464.9	1449.7	1183.3	1696.9	2628.7	3912.1	9063.0
Stargunner	697.0	9528.0	34081.0	14919.2	58946.0	58365.0	61582.0	51959.0
Surround	-9.7	5.4			-5.3	1.9	5.9	-0.9
Tennis	-21.4	-6.7	-2.3	-0.7	-2.3	-7.8	-5.3	-2.0
Time Pilot	3273.0	5650.0	5640.0	8267.8	5391.0	6608.0	5963.0	7448.0
Tutankham	12.7	138.3	32.4	118.5	96.5	92.2	56.9	33.6
Up'n Down	707.2	9896.1	3311.3	8747.7	16626.5	19086.9	12157.4	29443.7
Venture	18.0	1039.0	54.0	523.4	110.0	21.0	94.0	244.0
Video Pinball	20452.0	15641.1	20228.1	112093.4	214925.3	367823.7	295972.8	374886.9
Wizard of Wor	804.0	4556.0	246.0	10431.0	2755.0	6201.0	5727.0	7451.0
Yars' Revenge	1476.9	47135.2			6626.7	6270.6	4687.4	5965.1
Zaxxon	475.0	8443.0	831.0	6159.4	5901.0	8593.0	9474.0	9501.0

Table 7: Raw scores obtained on the original 49 Atari games plus 8 additional games where available, evaluated on human starts. Human, random, DQN and tuned Double DQN scores are from van Hasselt et al. (2016). Note that the Gorila results from Nair et al. (2015) used much more data and computation, but the other methods are more directly comparable to each other in this respect.

Game			DQN			Double DQN (tuned)		
	random	human	baseline	Gorila	rank-b.	baseline	rank-b.	prop.
Alien	128.3	6371.3	570.2	813.5	1191.0	1033.4	1334.7	900.5
Amidar	11.8	1540.4	133.4	189.2	98.9	169.1	129.1	218.4
Assault	166.9	628.9	3332.3	1195.8	3081.3	6060.8	6548.9	7748.5
Asterix	164.5	7536.0	124.5	3324.7	9199.5	16837.0	22484.5	31907.5
Asteroids	871.3	36517.3	697.1	933.6	1677.2	1193.2	1745.1	1654.0
Atlantis	13463.0	26575.0	76108.0	629166.5	207526.0	319688.0	330647.0	593642.0
Bank Heist	21.7	644.5	176.3	399.4	823.7	886.0	876.6	816.8
Battlezone	3560.0	33030.0	17560.0	19938.0	22250.0	24740.0	25520.0	29100.0
Beam Rider	254.6	14961.0	8672.4	3822.1	12041.9	17417.2	31181.3	26172.7
Berzerk	196.1	2237.5			644.0	1011.1	865.9	1165.6
Bowling	35.2	146.5	41.2	54.0	58.0	69.6	52.0	65.8
Boxing	-1.5	9.6	25.8	74.2	69.6	73.5	72.3	68.6
Breakout	1.6	27.9	303.9	313.0	481.1	368.9	343.0	371.6
Centipede	1925.5	10321.9	3773.1	6296.9	2959.4	3853.5	3489.1	3421.9
Chopper Command	644.0	8930.0	3046.0	3191.8	6685.0	3495.0	4635.0	6604.0
Crazy Climber	9337.0	32667.0	50992.0	65451.0	109337.0	113782.0	127512.0	131086.0
Defender	1965.5	14296.0			20634.0	27510.0	23666.5	21093.5
Demon Attack	208.3	3442.8	12835.2	14880.1	19478.8	69803.4	61277.5	73185.8
Double Dunk	-16.0	-14.4	-21.6	-11.3	-5.3	-0.3	16.0	2.7
Enduro	-81.8	740.2	475.6	71.0	1265.6	1216.6	1831.0	1884.4
Fishing Derby	-77.1	5.1	-2.3	4.6	3.5	3.2	9.8	9.2
Freeway	0.1	25.6	25.8	10.2	28.4	28.8	28.9	27.9
Frostbite	66.4	4202.8	157.4	426.6	288.7	1448.1	3510.0	2930.2
Gopher	250.0	2311.0	2731.8	4373.0	17478.2	15253.0	34858.8	57783.8
Gravitar	245.5	3116.0	216.5	538.4	351.0	200.5	269.5	218.0
H.E.R.O.	1580.3	25839.4	12952.5	8963.4	15150.9	14892.5	20889.9	20506.4
Ice Hockey	-9.7	0.5	-3.8	-1.7	-3.8	-2.5	-0.2	-1.0
James Bond 007	33.5	368.5	348.5	444.0	1074.5	573.0	3961.0	3511.5
Kangaroo	100.0	2739.0	2696.0	1431.0	9053.0	11204.0	12185.0	10241.0
Krull	1151.9	2109.1	3864.0	6363.1	11209.5	6796.1	6872.8	7406.5
Kung-Fu Master	304.0	20786.8	11875.0	20620.0	20181.0	30207.0	31676.0	31244.0
Montezuma's Revenge	25.0	4182.0	50.0	84.0	44.0	42.0	51.0	13.0
Ms. Pac-Man	197.8	15375.0	763.5	1263.0	964.7	1241.3	1865.9	1824.6
Name This Game	1747.8	6796.0	5439.9	9238.5	8738.5	8960.3	10497.6	11836.1
Phoenix	1134.4	6686.2			16107.8	12366.5	16903.6	27430.1
Pitfall!	-348.8	5998.9			-193.7	-186.7	-427.0	-14.8
Pong	-18.0	15.5	16.2	16.7	18.7	19.1	18.9	18.9
Private Eye	662.8	64169.1	298.2	2598.6	2202.3	-575.5	670.7	179.0
Q*bert	183.0	12085.0	4589.8	7089.8	12740.5	11020.8	9944.0	11277.0
River Raid	588.3	14382.2	4065.3	5310.3	10205.5	10838.4	11807.2	18184.4
Road Runner	200.0	6878.0	9264.0	43079.8	57207.0	43156.0	52264.0	56990.0
Robotank	2.4	8.9	58.5	61.8	51.3	59.1	56.2	55.4
Seaquest	215.5	40425.8	2793.9	10145.9	11848.8	14498.0	25463.7	39096.7
Skiing	-15287.4	-3686.6			-29404.3	-11490.4	-10169.1	-10852.8
Solaris	2047.2	11032.6			134.6	810.0	2272.8	2238.2
Space Invaders	182.6	1464.9	1449.7	1183.3	1696.9	2628.7	3912.1	9063.0
Stargunner	697.0	9528.0	34081.0	14919.2	58946.0	58365.0	61582.0	51959.0
Surround	-9.7	5.4			-5.3	1.9	5.9	-0.9
Tennis	-21.4	-6.7	-2.3	-0.7	-2.3	-7.8	-5.3	-2.0
Time Pilot	3273.0	5650.0	5640.0	8267.8	5391.0	6608.0	5963.0	7448.0
Tutankham	12.7	138.3	32.4	118.5	96.5	92.2	56.9	33.6
Up'n Down	707.2	9896.1	3311.3	8747.7	16626.5	19086.9	12157.4	29443.7
Venture	18.0	1039.0	54.0	523.4	110.0	21.0	94.0	244.0
Video Pinball	20452.0	15641.1	20228.1	112093.4	214925.3	367823.7	295972.8	374886.9
Wizard of Wor	804.0	4556.0	246.0	10431.0	2755.0	6201.0	5727.0	7451.0
Yars' Revenge	1476.9	47135.2			6626.7	6270.6	4687.4	5965.1
Zaxxon	475.0	8443.0	831.0	6159.4	5901.0	8593.0	9474.0	9501.0

表 7: 原始 49 款 Atari 游戏以及 8 款可用的附加游戏获得的原始分数, 根据人类开始进行评估。人类、随机、DQN 和调整的双 DQN 分数来自 van Hasselt 等人。(2016)。请注意, 大猩猩是 Nair 等人的结果。(2015) 使用了更多的数据和计算, 但其他方法在这方面彼此之间具有更直接的可比性。